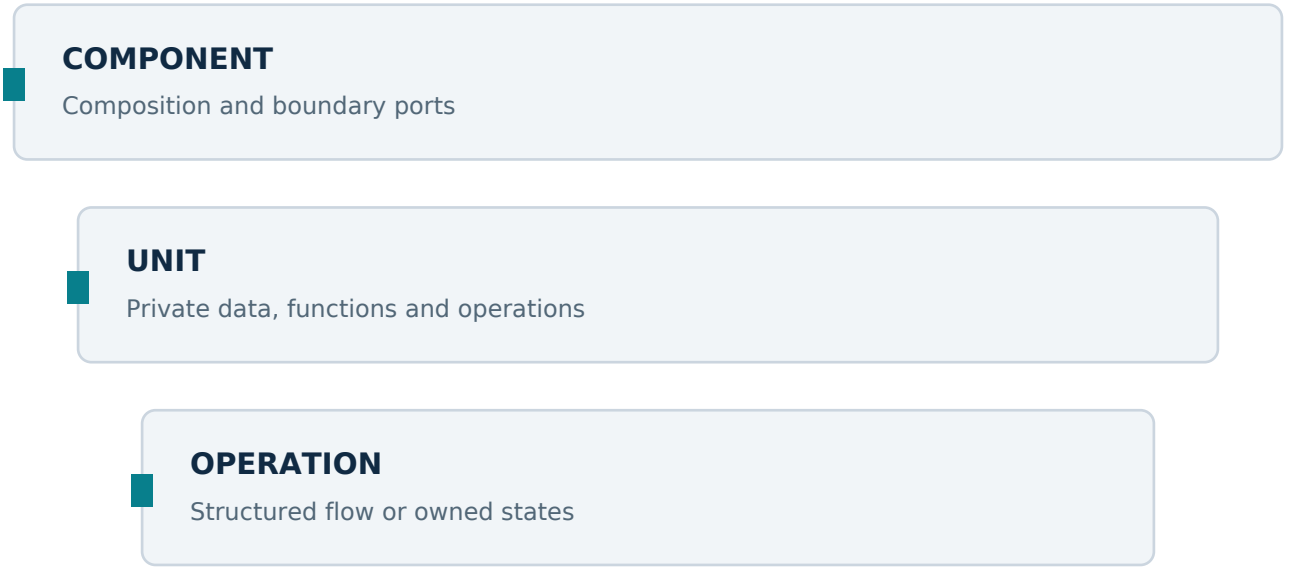
SOFTWARE ARCHITECTURE MODELING LANGUAGE

SWAML

Complete language specification

Release 0.1.0 / Language 0.1 / Draft revision 0.1.1



Typed interfaces at every software boundary.

Embedded realtime systems, services and event-driven architectures.

14 October 2026 / Catalin Ilie and SWAML contributors

Independent language design on a proposed KerML semantic foundation.

github.com/voidaliot/swaml

About this edition

RELEASE 0.1.0 / COMPLETE SPECIFICATION BOOK

This book assembles the complete SWAML 0.1 normative document set and its supporting guides from the repository. Release 0.1.0 contains specification/reference-core revision 0.1.1, including the completed specification audit. Source models use `swaml "0.1";`. These identifiers describe different versioned artifacts.

Normative chapters define the language. Informative chapters provide tutorials, rationale and tooling guidance. The concrete grammar is normative; model examples are informative. This is an independently authored draft, not an OMG UML release, an AUTOSAR implementation or a certified KerML tool.

The implementation supplies a parser, language server foundation and broad static checking. Kernel export, execution, analysis evidence and graphical editors remain separate implementation milestones. The status and conformance chapters qualify every claim.

Navigation	How to use this PDF
Contents and bookmarks	Select a contents entry or use the PDF outline to open a chapter or section.
Cross-references	Links to included documents jump inside this book. Other repository links target release v0.1.0.
Code and tables	Code is selectable, numbered and wrapped without omitting source text. Tables repeat their headers across pages.
Source record	The build manifest lists every included source and its SHA-256 digest. Original source files are embedded as PDF attachments. The release includes a PDF checksum.

Original text and code: MIT license, copyright 2026 Catalin Ilie. Referenced standards retain their own terms; no standards PDFs are redistributed. Full license text appears at the end.

Contents

About this edition 2

SWAML language specification 7

1. Normative document set . . . 7
2. Normative terms and precedence . . . 7
3. Design invariants . . . 8
4. Abstract model inventory . . . 8
5. Conformance summary . . . 9

1. Structure, names and types 10

1.1 Lexical model . . . 10
1.2 Namespaces and resolution . . . 10
1.3 Definitions and occurrences . . . 10
1.4 Value types . . . 11
1.5 Expression rules . . . 12
1.6 Requirements for tools . . . 12

2. Interfaces and communication 13

2.1 Contract identity and members . . . 13
2.2 Ports and boundary exposure . . . 13
2.3 Emission and reception ownership . . . 14
2.4 Exchange semantics . . . 14
2.5 Queue and delivery policy . . . 15
2.6 Timing of messages . . . 16

3. Operations, flow and states 17

3.1 Operation identity and activation . . . 17
3.2 Flow execution . . . 17
3.3 Scheduling and concurrency . . . 18
3.4 Stateful operations . . . 19
3.5 Transition triggers and selection . . . 19
3.6 Transition execution . . . 19

4. Systems, timing, cases and conformance 21

4.1 Systems and profiles . . . 21
4.2 Deployment . . . 22
4.3 Requirements and cases . . . 22
4.4 View declarations . . . 23
4.5 Static validity rule groups . . . 23
4.6 Conformance levels . . . 24

5. KerML semantic mapping 25

5.1 Foundation and boundary . . . 25
5.2 Semantic specializations . . . 25
5.3 Normalization algorithm . . . 26
5.4 Required extension predicates . . . 27
5.5 Interchange and round-trip obligations . . . 27

6. Operational contract and target bindings 28

6.1 Configuration and observable occurrences . . . 28
6.2 Initialization and release . . . 28
6.3 Dispatch, backlog and snapshots . . . 29
6.4 Queue offers and fanout . . . 29
6.5 RPC completion and races . . . 30
6.6 State reaction ordering . . . 31
6.7 Execution binding contract . . . 31

7. Normalized semantic model 32

7.1 Identity, ownership and source trace . . . 32
7.2 Declaration graph and links . . . 32
7.3 Contract projection and route normalization . . . 33
7.4 Executable normalization . . . 34
7.5 Kernel representation and SWAML library responsibilities . . . 34
7.6 Case and view normalization . . . 35
7.7 Equivalence and completeness . . . 35

8. Conformance obligations and evidence 36

8.1 Static acceptance . . . 36
8.2 Runtime obligations . . . 37
8.3 Analysis claims . . . 38
8.4 Kernel, editor and graphical acceptance . . . 38
8.5 Scope and future evolution . . . 39

SWAML graphical notation, version 0.1 40

1. Views and navigation . . . 40
2. Visual alphabet . . . 41
3. Lines and direction . . . 41
4. Flow View . . . 42
5. Sequence and case views . . . 43
6. Grid View . . . 43
7. Synchronization, layout and editor safety . . . 43

A. Complete concrete grammar 45

B. Grammar-to-semantics inventory 47

C. Model examples 52

01 hello . . . 52
02 realtime controller . . . 53
03 service pubsub . . . 54
04 stateful operation . . . 56
05 values and flow . . . 57
06 mixed gateway . . . 58
07 client server . . . 60
08 service field . . . 61

SWAML 0.1 user guide 63

1. Install and check a model . . . 63
2. Write the contract first . . . 63
3. Choose component and unit boundaries . . . 64
4. Describe operation control flow . . . 64
5. Model a service call and its failure . . . 65
6. Use publish/subscribe without a hidden event bus . . . 65
7. Put states inside an operation . . . 65
8. Model realtime constraints . . . 66
9. Separate deployment from architecture . . . 66
10. Organize a larger model . . . 66
11. Navigate through views . . . 66
12. Resolve diagnostics . . . 67

Service fields, repeated consumers and checked traces . . . 67

Architecture analysis and SWAML design rationale 68

Comparison . . . 68
AUTOSAR Classic . . . 68
AUTOSAR Adaptive . . . 69
UML . . . 69
SysML 2.0 and KerML 1.0 . . . 69
Additional architecture lessons . . . 69
Mapping examples and limits . . . 70
Decisions recorded for 0.1 . . . 70
Source handling . . . 71

Language tooling and integration 72

Package contract . . . 72
Reproducible commands . . . 72
Services and source ownership . . . 72
Language-server integration . . . 73
Browser hosts . . . 73
Graphical architecture . . . 73

Kernel backend boundary . . . 73
Validation and release gates . . . 74

Implementation status - 0.1.1 **75**
Implemented and tested . . . 75
Limits of static acceptance . . . 76
Specified, not implemented . . . 76
Intentionally outside 0.1 . . . 77

Specification completeness audit - 0.1.1 **78**
Findings and resolutions . . . 78
Completion checklist . . . 78
Remaining implementation work . . . 79

ADR 0001 - Ownership and communication boundaries **80**

Decision 0002: Complete semantics before claiming complete tooling **81**

PDF specification and release publication **82**
Version identifiers . . . 82
Rebuild the book . . . 82
Quality and release gate . . . 82

Contributing to SWAML **83**
Development setup . . . 83
Propose a language change . . . 83
Implement and test . . . 83
Pull requests and review . . . 83
Documentation and standards . . . 84
Resolve specification issues . . . 84
Report a defect . . . 84

Instructions for agents working on SWAML **85**
Read first . . . 85
Preserve the language invariants . . . 85
Source and architecture . . . 85
Required checks . . . 85
Working practices . . . 86

D. External references **87**

E. License **88**

NORMATIVE / LANGUAGE CONTRACT

SWAML language specification

Version: 0.1.1 · **Language header:** `swaml "0.1";` · **Status:** complete 0.1 draft for implementation and review · **Date:** 2026-10-14

SWAML means Software Architecture Modeling Language. This document and its normative chapters define a software architecture language over a KerML-based semantic model. SWAML is independently authored; “unofficial UML 3.0” describes its motivation and does not denote an OMG standard or compatibility level.

1. Normative document set

Read these chapters together. Their requirements have equal standing:

1. Structure, names and types.
2. Interfaces and communication.
3. Operation and state semantics.
4. Timing, deployment, cases and conformance.
5. KerML semantic mapping.
6. Graphical notation.
7. Concrete grammar.
8. Operational contract and target bindings.
9. Normalized semantic model.
10. Conformance obligations and evidence.
11. Machine-readable mapping inventory.

The user guide, architecture comparison, examples and design decisions are informative. The implementation matrix reports current tool support; a missing implementation does not waive a language rule.

2. Normative terms and precedence

MUST and MUST NOT state conformance requirements. SHOULD states a recommendation with an explicitly documented justification needed for deviation. MAY grants permission. “Error” means a model is not valid under this version. An implementation may retain an invalid model for editing but MUST NOT report it as fully valid.

The grammar determines tokens and parse structure. Prose determines validity and meaning. If grammar accepts text that violates a semantic rule, that text is invalid. If a discovered conflict makes the two impossible to satisfy, record a specification defect; do not silently choose a runtime meaning. Unknown syntax and unknown language versions MUST be rejected.

The language is closed at version 0.1: constructs not listed in the grammar are not implicitly inherited from UML, SysML or KerML. Definitions may be reused by instantiation, but inheritance, generics, redefinition, unrestricted KerML embedding and dynamic creation are not part of this version. This constraint makes the whole v0.1 language implementable without pretending to reproduce three other standards.

3. Design invariants

- **I1 - Boundary:** every exchange between distinct unit instances crosses port instances typed by the same interface contract and a permitted connector/delegation path.
- **I2 - Ownership:** components own composition; units own private data, functions and operations; operations own statement flow or states. No performer element, behavior allocation, free action, free signal, or global variable exists.
- **I3 - Event provenance:** every outgoing data/event member of a unit port has at least one syntactically associated emitting operation. Every incoming member has exactly one handling operation, including a stateful operation where applicable.
- **I4 - Type safety:** payloads and results conform to nominal contracts. Messages transfer values, never aliases to another unit's private state.
- **I5 - Explicit communication policy:** every connector declares delivery, ordering, capacity and overflow. Time-sensitive connections add latency/freshness bounds.
- **I6 - Views preserve meaning:** views reference existing semantic elements. Drawing a message cannot create a hidden communication channel.

Local calls, calculation values, timer expiration, initial activation and state completion do not constitute exchanges between units. They are the only exceptions needed for local computation and lifecycle. An external actor or device that exchanges model data MUST be represented through a component/unit port; a case actor label cannot bypass I1.

4. Abstract model inventory

Element	Owns	References / meaning
Model	packages	exact language version
Package	imports, named elements	namespace
DataType / EnumType	fields / literals	immutable value shape
Interface	data, event or operation members	nominal versioned contract
Component definition	ports, child instances, connectors, delegation	reusable composition type
Unit definition	ports, private variables, operations, functions	reusable behavior/data boundary
Instance	no inline definitions in 0.1	one component or unit definition
Port	role and optionality	one interface
Connector	QoS	source and target port paths
Delegation	no data conversion	outer port and immediate child port
Operation	parameters, result, trigger, timing; flow **or** machine	executable behavior owned by a unit
Function	parameters, result, expression	pure local calculation
Machine	states, transitions, initial reference	lifetime of a stateful operation

Element	Owns	References / meaning
State	entry and exit blocks	one operation-local condition
Transition	guard, effect, priority, trigger	source and target states
System	component instances, connectors, execution nodes, deployment	closed assembly and profile
Requirement	text	named obligation
Case	exchange steps, trace links	one system and requirements
View	kind	one subject, no runtime semantics

All named elements have stable identity within a model version. Tools SHOULD persist a separate UUID identity map for editing; the language's normative textual identity is its qualified declaration/instance path. A rename is an identity-preserving edit only when the tool updates that map and all references atomically.

5. Conformance summary

A parser supports concrete syntax. A semantic validator additionally enforces static rules. A KerML mapper additionally produces the normalized representation in chapter 5. An execution backend additionally realizes chapter 3 and communication policies. A graphical editor additionally satisfies the view/editing contract. A tool MUST state which levels it implements; success at one level MUST NOT imply the others.

This repository supplies a parser, LSP foundation and a documented subset of semantic validation. The specification defines more than that implementation, especially runtime delivery, idempotency, target snapshot realization, timing proof, interchange and graphical editing. See the exact support matrix.

NORMATIVE / LANGUAGE CONTRACT

1. Structure, names and types

Normative chapter of the SWAML 0.1 specification.

1.1 Lexical model

Source files MUST use UTF-8 and the `.swaml` extension. Whitespace and `//` line comments and `/* … */` block comments are insignificant outside strings. Block comments do not nest. Identifiers match `[_a-zA-Z][a-zA-Z0-9_]*` and are case sensitive. Every quoted word in the grammar is reserved. This version has no escaped identifiers.

Strings use double quotes and backslash escapes. A conforming implementation MUST support `\"`, `\\`, `\n`, `\r`, and `\t`; other escape sequences are invalid in conforming source. Raw control characters U+0000-U+001F, including line breaks and tabs, MUST NOT occur inside strings; use the permitted escapes. The lexer enforces this portable subset.

Unsigned decimal integers and decimal real literals are supported; a minus sign is a unary operator. Decimal real literals require digits on both sides of the point. Hexadecimal, exponent and suffix forms are absent. Duration literals consist of a positive integer followed by `ns`, `us`, `ms` or `s`. A duration MUST normalize to an exact integer nanosecond count no larger than 2^53-1. Time has no implicit unit.

The document grammar is `swaml STRING ;` followed by zero or more packages. Only the header value `0.1` is supported. An empty model is valid as an editing/library artifact, but contains no closed system.

1.2 Namespaces and resolution

Packages are top-level and non-nested in 0.1. A package name MUST be unique in the loaded model workspace. A package is declared in one file, although one file may contain several packages. Named elements in the same namespace MUST be unique; overload resolution is not supported.

`A::Type` identifies a package declaration. `import A::*;` exposes A's top-level declarations by short name in the importing package. Imports are not transitive. A short reference MUST resolve to exactly one declaration across the local package and its direct imports. Ambiguity is an error even when one candidate is local. Fully qualified references do not require an import.

Periods express containment/feature paths, not package qualification. A structural instance path follows actual child instances. A behavior communication path has exactly two identifiers: `localPort.member`. Directly following another unit's data or operation is forbidden. Pure record field paths follow only values already in the operation's local environment.

Local operation and function references resolve only in the owning unit. State names resolve only in the owning machine. Deployment nodes resolve only in the owning system. No lookup fallback may expose a similarly named declaration from another unit.

1.3 Definitions and occurrences

`component def C` and `unit def U` declare reusable types. `component c: C;` and `unit u: U;` declare contained occurrences. The keyword MUST agree with the referenced definition kind. An instance has multiplicity exactly one in this version. Repeated instances require separate declarations.

A component owns ports, unit/component instances, connectors and delegation. A unit owns ports, private variables, functions and operations. A system owns component instances, connectors, execution nodes and deployments. A system-level unit instance is syntactically recognizable but invalid; wrap it in a component to make the architecture boundary explicit.

The containment graph of component definitions MUST be acyclic. Every occurrence of a unit receives distinct variable storage and operation activations. Two instances of the same definition are distinct communication boundaries and still require ports between them.

Definitions are not runtime singletons. Defining an unused component or unit creates no runtime participant. Library definitions may leave their nonoptional ports unconnected until instantiated. A closed system MUST connect every nonoptional port, including boundary aliases, after resolving delegation. An optional leaf cannot weaken a required boundary, nor the reverse.

Unit state is private to its instance. A variable has a declared type and a pure initializer. Initializers MUST be independent of other mutable variables and all communication. Operation/function parameters and local variables MUST NOT shadow unit variables or other live values in their scope. Blocks create nested lexical scopes; a variable declared in a branch does not exist after that branch. Parameters are copied input values; assignment changes the local copy only.

1.4 Value types

Type	Domain / semantics
`Bool`	`true` or `false`
`Int`	exact signed integers between -(2^53-1) and 2^53-1
`Real`	finite IEEE 754 binary64 values, with nearest-even arithmetic
`String`	finite Unicode text; no implicit encoding or transport size guarantee
`Void`	no result value; only an operation/signature result, never a variable, parameter, field, array or pure-function result
`datatype`	a finite record with named typed fields
`enum`	a nominal set of one or more unique literals
`T[n]`	a fixed array of exactly n values, n a positive safe integer

Datatypes and enums are nominal: two declarations with identical members are distinct types. Recursive records, including recursion through arrays, MUST be rejected. There are no pointers, references, nullable values, dynamic collections, aliases, generics or inheritance in 0.1. Bounded strings and machine integer widths may be added by future target profiles; they are not inferred from these primitives.

`record T { field = expression; … }` MUST assign every field of T exactly once and no others. `enumValue T.Literal` MUST name a declared literal. `array [e1, e2, …]` constructs a nonempty fixed array; its elements MUST have one type, allowing numeric Int-to-Real promotion inside the literal. Arrays of differing lengths are incompatible.

Record fields may be read by `value.field.subfield`. `arrayValue[index]` reads one array element, with an Int index. Only one final index suffix is available in 0.1; a local variable can hold an intermediate array value. Bounds are zero inclusive to length exclusive. Out-of-bounds access fails the current operation at runtime and MUST NOT produce an undefined value. Whole-value assignment is supported; mutating a record field or array element in place is not part of 0.1.

Assignment, argument passing and return require the same nominal type and array length, with one exception: a scalar Int may widen to Real. Arrays do not implicitly widen. No numeric narrowing or conversion between enums and integers is implicit.

Numeric overflow, non-finite results, division by zero and invalid indexing fail execution. The parser/validator rejects out-of-range literal integers, but proving arithmetic ranges requires execution or additional analysis.

1.5 Expression rules

From lowest to highest precedence: `or`, `and`, `== !=`, `< <= > >=`, `+ -`, `* / %`, unary `not -`, primary expressions. Binary operators associate left. Parentheses override precedence. `and` and `or` short-circuit left-to-right. Other operands and call arguments are evaluated left-to-right.

Boolean operators require Bool operands. Ordered comparison and arithmetic require numeric operands. Equality compares equal nominal value types recursively (records by declared fields, arrays by ordered values), or two numerics after promotion. String equality is exact code-point equality with no normalization; no ordering or concatenation operator is defined. `%` is integer remainder with the sign of the dividend and requires Int operands. `/` always returns Real. Other arithmetic returns Real if either input is Real and Int otherwise.

A pure function has the syntax `function name(parameters): Type = expression;`. Its environment consists only of parameters, literals, record/enum/array construction and other functions in the same unit. It MUST NOT read unit variables, call operations, or communicate. Local call graphs MUST be acyclic in 0.1. `compute name(arguments)` evaluates the expression once with copied arguments and returns its value.

1.6 Requirements for tools

A reference that cannot be resolved, has the wrong target kind, or is ambiguous is an error. Tools MUST distinguish declaration paths, instance paths and local value paths even when their spelling is similar. A tool supporting rename MUST update cross-references and string-valued path references together; generic Langium rename alone is not sufficient for the latter in this prototype.

NORMATIVE / LANGUAGE CONTRACT

2. Interfaces and communication

Normative chapter of the SWAML 0.1 specification.

2.1 Contract identity and members

An interface declaration has a kind, package-qualified name, explicit `major.minor.patch` version string, and at least one member. Its identity is its declaration plus that version. Member names are unique across the entire interface; operation overloading is absent.

Kind	Permitted members	Port roles
`senderReceiver`	`data name: T sampled;` or `queued;`	`sender`, `receiver`
`clientServer`	operation signatures	`client`, `server`
`publishSubscribe`	event signatures	`publisher`, `subscriber`
`service`	operation and event signatures	`provides`, `requires`

An operation signature declares ordered input parameters, a result, and optionally `idempotent`. An event signature declares zero or more ordered payload parameters and no result. Data is one typed value. A notification without payload uses a zero-parameter event. No event may be declared directly in a package, unit, component or state.

A service field is modeled explicitly as a getter operation, optional setter operation and changed event. This avoids a second implicit communication mechanism. A setter returns an application result; its implementation owns the decision whether and when to publish the changed event. Property storage never becomes remotely writable data.

Version strings document compatibility intent: a breaking change SHOULD increase major; additive changes SHOULD increase minor; a meaning-preserving correction SHOULD increase patch. However, 0.1 performs **exact nominal matching**, not automatic version negotiation. Use a differently named declaration for another version in the same package and an adapter between versions. Adding an event may require a new receiver and is not automatically compatible with an existing complete implementation.

2.2 Ports and boundary exposure

A port belongs to one unit or component and references exactly one interface. Its role must be legal for that interface kind. `optional` permits the port to have zero connections in a closed system; it does not relax type, direction, handler or emission requirements.

A connector joins immediate child ports of its owner: `connect name: child.port -> child.port qos { … };`. Its owner is a component or system. Both paths MUST have exactly two segments and reference ports, not members, operations, variables, states or grandchildren. The interface declarations MUST be identical.

Legal connector orientations are sender → receiver, client → server, publisher → subscriber, and requires → provides. The last two roles support provider-to-consumer event traffic over the same service contract; the arrow denotes the request orientation, not a unidirectional transport restriction.

A component port MUST have exactly one delegation to an immediate child port: `delegate outer -> child.port;`. Delegation MUST preserve role and nominal interface. It is a forwarding boundary, with no

implicit queue, payload conversion, delivery retry or additional source. Child components can delegate again. To cross a hierarchy, expose a port at each boundary; no connector may tunnel to a grandchild.

At system closure, recursively expand instances, resolve each delegation to a leaf unit port, and inspect actual connector endpoints. Every nonoptional port, including boundary aliases, MUST resolve to a connected leaf. Receiver ports have exactly one source; client/requires ports have exactly one provider. Sender, publisher, server and provides ports can participate in multiple connectors. Subscriber ports may receive from multiple publishers. A duplicate connector resolving to the same directed leaf pair is an error; it must not create an accidental duplicate subscription. A component may not expose the same child port under two aliases in 0.1.

2.3 Emission and reception ownership

A unit's outgoing data/event member MUST occur in at least one `send`/`publish` statement inside one of its operations, including a state entry, exit or transition effect. This is a static provenance relationship, not a proof of reachability or a promise that the operation always emits.

Each incoming data/event member MUST have exactly one handler owner: an ordinary operation triggered by that member, or one stateful operation containing one or more receiving transitions for it. An operation signature on a server/provides port MUST have exactly one ordinary operation with `on call port.member`. Several transitions in one stateful operation count as one owner. Two different operations receiving the same member are invalid. To intentionally fan out processing, use a local coordinating handler that invokes helpers, or explicit subscriber units.

An ordinary handler MUST reproduce the member's ordered parameter types. Names may differ. A data handler has one parameter of the data type; an event handler has the event's parameter list. Both return Void. An RPC handler MUST also reproduce the signature's result type.

2.4 Exchange semantics

`send p.data(value);` emits one typed data value through a sender port. `publish p.event(args);` emits one typed event through a publisher/provides port. Both paths are exactly `localPort.member`. The transfer copies the value; a target implementation may use zero-copy storage only if it preserves isolation and observable value semantics.

For each outgoing permitted connector, the communication system independently offers the message to that connector's destination queue. Emission does not execute a remote handler on the caller's stack. All fanout offers are attempted. Any `reject` fails the emitting activation after those decisions; accepted offers cannot be rolled back. No interconnector offer order is implied. Programs needing atomic broadcast must model a protocol explicitly.

`request p.operation(args) as result within duration otherwise { … };` issues one correlated RPC to the single provider selected by its connector. The result name becomes available in the current block only after successful completion. The caller is suspended until a result or failure; this is an awaited request in the model, independent of whether generated code uses a blocking call or a future.

The communication system creates a fresh correlation identity for each request and carries its response back through the same interface/port/connector route. That response is the operation's declared result, not an independent event or unmodeled channel. A server return and a service event are distinct occurrences and have no implicit order beyond their operation's control flow.

The `otherwise` block executes on timeout, unavailable optional provider, admission rejection, transport failure or server `fail`. Its local `failure: String` is respectively `Timeout`, `Unavailable`, `Rejected`, `Transport`, or `RemoteFailure`. Remote implementation error strings are not automatically disclosed. Every path of the failure block MUST return from or fail the enclosing operation. No result is bound on failure. The implicit

failure name must not shadow a live value. A Void success name labels completion only and cannot be used as data.

The timeout starts at issuance and includes admission, queueing, service execution and return transport. Timeout does not cancel committed server effects. A late response to that correlation is discarded. Automatic application-level retry is absent. A backend using retries to satisfy delivery policy MUST preserve correlation and timeout budgets. Application-specific unsuccessful results remain typed results, not communication failures.

An outgoing optional unbound data/event port accepts an emission with no receivers; an unbound request fails as Unavailable. Incoming optional ports retain their handlers even if no external source is connected.

2.5 Queue and delivery policy

Every connector MUST specify all four base QoS fields, in the grammar's order:

Field	Meaning
`delivery bestEffort`	messages may be lost, no retransmission obligation; accepted unique transmissions are not deliberately duplicated
`delivery atMostOnce`	delivery may be absent, but a given logical message is presented to a handler no more than once
`delivery atLeastOnce`	admitted messages may be retried and duplicated; eventual delivery is required only under a live connection, available receiver and unexpired lifetime/budget
`ordering fifo`	per connector and per member, presented messages preserve sender emission order; duplicates may repeat an item before the next item
`ordering unordered`	presentation order is unspecified
`capacity n`	positive bound on buffered messages per connector, direction and member; executing activations are not in this queue
`overflow dropOldest`	remove the oldest buffered message and admit the new one
`overflow dropNewest`	discard the incoming message
`overflow reject`	refuse admission and signal the emitting activation

Overflow and expiration govern admission/retention and may intentionally discard messages even when delivery is atLeastOnce. This policy is therefore not a promise of lossless application execution. Reliability is separate from handler success; processing that fails is not silently repeated. AtLeastOnce RPC is permitted only when every operation on the interface is declared idempotent. A runtime/implementation MUST substantiate idempotency (including event side effects); the declaration alone is not a proof.

Exactly-once delivery is deliberately not a QoS option. Application-level deduplication and transactional processing can be modeled as operations and state, but depend on storage and failure assumptions outside the connector.

Sampled data retains only the latest pending value for that member. Any connector on an interface containing sampled members MUST use capacity 1 and dropOldest; this single connector policy also applies to other members of that interface. Use separate interfaces if mixed buffering policies are needed. Queued data preserves individual samples subject to explicit capacity/overflow. Receiving a sampled value is an activation with that value; there is no hidden cross-unit shared variable or implicit read operation.

Subscriptions are active while their endpoint units are alive. Pub/sub does not require a central broker; a logical connector describes the allowed subscription. Broker placement and behavior can be modeled

explicitly. Runtime discovery, topic wildcards, content filters, retention persistence and dynamic subscription management are outside 0.1 and MUST NOT be assumed. Use explicit alternate systems or adapter components to document such deployment variants.

2.6 Timing of messages

`maxLatency` is a bound from accepted emission to availability for destination dispatch, excluding handler execution. `maxAge` is a maximum age since the producing emission timestamp; expired messages are discarded before dispatch. Clock translation across nodes is an implementation obligation. If a backend cannot establish a sufficiently bounded common time basis, it cannot claim the corresponding bound.

Request and response each have their own directional queue. A service event has its provider-to-consumer event queue. The connector QoS applies to each independently. Capacity is not shared between request, response and event traffic. A timed-out correlation does not keep its response queue alive indefinitely.

`maxLatency` and `maxAge` are obligations, not simulated delay values. Violations must be observable as runtime contract violations by a conforming checking executor. A declaration is not a measured proof. Security, authorization, integrity, serialization, encryption and transport binding belong to a target implementation profile and MUST preserve these interface boundaries. Chapter 6 defines admission, timeout races, lifetime and failure details.

NORMATIVE / LANGUAGE CONTRACT

3. Operations, flow and states

Normative chapter of the SWAML 0.1 specification.

3.1 Operation identity and activation

An operation belongs directly to a unit definition and is instantiated per unit instance. It has a name, ordered parameters, result type, optional trigger, optional timing contract, and exactly one body form: a statement block or an owned state machine. A component does not execute operations; execution lives in its contained units.

Trigger	Activation and permitted signature
omitted	local `invoke`; the callable operation must return Void
`on manual`	parameterless Void root activated by a test/administrative harness, carrying no model data
`on start`	parameterless Void operation, once after unit initialization and connector readiness
`on every D`	parameterless Void operation released at unit start + kD, for positive integer k
`on receive p.m`	copied data/event arguments, Void result
`on call p.m`	copied RPC arguments and exact contract result

Manual activation is a control of the modeling environment, not a way to inject an external data exchange without a port. A modeled UI command, interrupt payload, device sample, network event or external client MUST use an interface. Timers and initial activation are local lifecycle mechanisms.

Several ordinary operations may exist in one unit. There is no UML-style separate “method implementation of action definition” layer. The interface signature describes an RPC contract, while the unit operation triggered by that member is its implementation. A stateful operation directly contains its states. Reuse of algorithms is through functions or local operations, and reuse of architecture is through definitions and instances.

3.2 Flow execution

An activation holds its copied parameters, locals, a program position, its unit reference and any request correlation. Execute statements in source order. Blocks introduce lexical scope and discard block-local variables on leaving the block.

Statement	Semantics
`let x: T = e;`	evaluate e, typecheck, create a local mutable value
`set x = e;`	replace an existing local/parameter/unit-variable value; no field or remote assignment
`invoke op(args);`	execute an untriggered local Void operation synchronously; return resumes the caller
`return;` / `return e;`	complete the current operation; return a copied value when required
`if (e) { … } else { … }`	evaluate a Bool once and execute the selected block; else is optional

Statement	Semantics
`while (e) bound n { … }`	evaluate the Bool before each iteration; at most n body executions
`assert e;`	require a Bool true; false fails the current activation
`fail "reason";`	terminate the activation with a local failure reason
`send` / `publish` / `request`	execute chapter 2's interface exchanges

Reaching the end of a Void operation completes it. A value-returning operation MUST return a value or fail on every path. If the loop condition remains true after n iterations, execution fails with a bound violation; the nth iteration is allowed. Local recursion is forbidden for both operations and functions. `invoke` cannot call a triggered operation, a stateful operation, or a non-Void operation. Use functions for side-effect-free returned values, and a local Void operation plus owned data for local imperative work.

Failure unwinds the current activation and its synchronous local callers. It does not undo earlier variable assignments, published messages, responses, or external effects. For an RPC handler, failure produces a RemoteFailure response through the originating interface. For a periodic/start/manual/receive activation it produces a local diagnostic occurrence, not an implicit broadcast. To notify another unit, model a fault event and publish it explicitly on a declared interface in an explicit branch before failing.

Conditionals, bounded loops, expression evaluation and sequential local calls are the full flow language in 0.1. There is no generic fork/join statement, exception hierarchy, arbitrary host-language body or independent action node. Concurrent architectural behavior is expressed by separately activated operations and connected units. A later fork/join extension must define data-race and cancellation semantics before entering the grammar.

3.3 Scheduling and concurrency

All ordinary operations and state-machine reactions are serial by default. The unit has one logical serial dispatch lock. A serial activation holds this lock until completion, including while suspended on a request; the logical lock is not required to consume a physical thread. A local invoke borrows its caller's lock and does not acquire a second lock.

A callback to the same unit while a serial request is pending cannot execute until the serial activation ends. Cycles of requests can therefore time out or deadlock absent timeouts; all requests have finite timeouts. The language does not silently release the lock and allow reentrant state mutation. Authors SHOULD avoid synchronous request cycles.

`concurrency reentrant` permits overlapping activations with separate local frames. Such an operation MUST NOT mutate unit data directly or through local helpers, and MUST NOT invoke an operation that mutates shared state. It may read the unit's state snapshot captured at dispatch, and functions receive copied parameters. A conforming backend must preserve that snapshot or reject the model if its target cannot realize it. The reference validator follows local helper calls to check this prohibition. Snapshot realizability remains a backend obligation.

Input queues are per connector/member. FIFO constrains order within one queue. Across independent ready queues, the scheduler may choose any ready activation consistent with deadlines and the target profile; no global total order is inferred. Periodic activations released while the serial unit is busy remain pending; a target profile MUST bound that activation backlog; exceeding the bound records ActivationOverflow and stops the system, as defined in chapter 6. Deadlines are measured from release, not eventual dispatch. Merely listing WCET/deadline does not define thread priorities.

The core semantics is therefore a set of allowed execution traces. An execution backend may add a deterministic scheduling policy that narrows this set but must document it. A model requiring determinism across sources must explicitly order exchanges or provide a coordinating operation.

3.4 Stateful operations

A stateful operation MUST have no parameters, result Void, and `on start`. It owns one flat state machine with one or more states, exactly one initial reference and zero or more transitions. State and transition names are unique in the machine namespace. Hierarchical/orthogonal states, history pseudostates, entry/exit points and state inheritance are absent in 0.1.

Starting the operation enters its initial state and executes that state's entry block once. The operation remains alive between reactions without holding the serial unit lock. Each entry/exit/transition reaction is scheduled as a serial activation unless its timing contract explicitly says reentrant; a stateful operation nevertheless MUST have at most one reaction in progress and cannot reenter its own state machine.

A state owns an optional entry block and optional exit block. A `final` state completes the operation after its entry finishes and MUST have no outgoing transition. Ordinary completion is not an event emitted to another unit. A notification on completion must be published explicitly before completing.

3.5 Transition triggers and selection

A transition names source and destination states in the same machine. It has one trigger and optional Bool guard and nonnegative safe integer priority. Missing priority means 0; larger values take precedence.

- `on receive port.member` consumes one incoming data/event for the machine. Event parameter names are available as copied values in the guard and effect. Data has the local name `payload`. These names MUST NOT collide with unit variables. RPC operations are not allowed as machine transition triggers.
- `on after D` becomes eligible after D elapsed time since completing the source state's entry block. Leaving the source cancels its timer. Reentering starts a fresh timer.
- `on completion` is eligible after the state's entry and any prior reaction have finished, before waiting for a new external message.

For one stimulus, first select transitions from the current state with matching trigger, then evaluate guards, then choose the highest-priority enabled transition. Potentially competing transitions with the same source and identical trigger MUST have distinct priorities even when an author believes their guards are mutually exclusive. A missing guard means true. Unrelated queued stimuli are dispatched one at a time, in the scheduler order permitted by 3.3.

A received message with no enabled transition is consumed and ignored. A timer with no enabled transition expires once and is not retriggered until reentry. Completion transitions are reevaluated after each successful state entry and each machine reaction, including an ignored stimulus. Models MUST NOT contain unconditional completion cycles. Guarded cycles must reach quiescence; a checking executor MUST fail a reaction after 1,024 completion microsteps rather than spin forever. This limit is a semantic protection, not a proof of useful behavior.

3.6 Transition execution

A selected transition executes atomically with respect to this unit's other serial reactions:

1. Consume the matching stimulus, if any, and cancel all timers for the source state's entry generation.
2. Run the source state's exit block.
3. Run the transition effect with the stimulus payload in scope.
4. Update the current-state identity to the destination.

5. Run the destination state's entry block.
6. Complete if final; otherwise evaluate completion transitions or await a stimulus.

Entry and exit blocks do not inherit event parameters; only the guard/effect does. Copy a needed payload to unit data in the effect if entry behavior needs it. A self-transition exits and reenters, including entry/exit effects and resetting timers.

`return;` in any state block terminates the owning stateful operation immediately and cancels pending timers; remaining transition phases are skipped. `fail` terminates it with failure and also cancels timers. Neither rolls back prior side effects. Other queued messages still belong to the unit, but a terminated machine no longer dispatches its members; a target runtime must record these as undeliverable until unit restart.

Every state MUST be reachable from the initial state in the transition graph, ignoring guards for this static check. Reachability through satisfiable guards and eventual completion are stronger analysis obligations, not consequences of graph reachability. No state, operation or component may use an unbound global event to change another unit's behavior.

The operational contract further defines initialization barriers, snapshots, timer generations, stimulus ties, failure and resource bounds.

NORMATIVE / LANGUAGE CONTRACT

4. Systems, timing, cases and conformance

Normative chapter of the SWAML 0.1 specification.

4.1 Systems and profiles

A system is a closed architecture assembly. Its top-level occurrences are components. Several systems in one package may describe alternative assemblies of the same definitions. Instantiation recursively creates children and local variable storage, resolves delegation and connectors, and then enables startup/periodic activation. No unit may publish during a partially connected startup phase.

The system profile is `service`, `realtime` or `mixed`:

Obligation	service	realtime	mixed
Explicit connector QoS	required	required	required
Operation WCET/deadline/concurrency	optional	required for all used operations	required for all used operations
Connector maxLatency	optional	required	required
Sampled data maxAge	optional	required	required
Interface isolation and handler checks	required	required	required
Production scheduling evidence	target dependent	required for hard realtime claims	required for realtime claims

The mixed profile is intentionally conservative: all used operations and connectors carry timing contracts. It does not infer that a cloud-facing operation has no timing requirement. A future subassembly profile can relax this with explicit partitioning; no such implicit relaxation exists in 0.1.

A timing contract declares positive `wcet`, positive `deadline`, and `concurrency serial|reentrant`. WCET MUST be no larger than deadline. For periodic operations, deadline MUST be no larger than the period. A stateful operation's timing contract applies separately to each run-to-completion reaction, including entry/exit/effect and any completion microsteps, not to the lifetime of the machine.

WCET is active CPU execution time for the activation including synchronous local helpers. Deadline is elapsed time from activation release to completion, including waiting and interference. Connector maxLatency excludes destination dispatch wait. For a chain of operations, the end-to-end bound must account for release delay, queue delay, interference, transport, WCET and response delay; simply summing WCET is insufficient.

A conforming validator checks arithmetic consistency. A conforming target backend must additionally prove or measure its declared timing assumptions with an appropriate execution model. The core specifies no rate-monotonic/EDF policy, task mapping, interrupt priorities, memory size, hardware access mechanism or clock synchronization algorithm. Such a backend MUST supply a named external deployment profile, and MUST NOT claim timing conformance without evidence for those obligations.

4.2 Deployment

An execution node is named and has kind `mcu`, `process` or `container`. A deployment maps an existing component or descendant unit/component instance path onto a node declared in the same system. It does not reference a definition. The same explicit instance path may occur in only one deployment.

A component deployment applies to all its descendants by default. A deployment of a more specific descendant overrides that inherited location for its subtree. If no deployment exists for an instance or ancestor, the instance is logically modeled but unallocated. An unallocated architecture can be valid; an executable target configuration cannot silently pick a node and call it specified.

Deployment does not change interfaces, connector identity, callback ownership or private-data visibility. Interprocess or intermachine implementations must preserve value and correlation semantics. Serialization formats, discovery identifiers, service addresses, credentials, broker topics and physical channels belong to a separate target binding document. Any exported backend MUST record the SWAML element identity for every generated mapping.

4.3 Requirements and cases

`requirement Name "text";` declares a named natural-language obligation. `case Name for System satisfies Requirement, … { … }` defines a scenario and optional trace links. A `satisfies` link is an assertion of intended coverage, not verified fulfillment. Verification evidence belongs to a test, analysis or review artifact linked by an implementation tool.

A case step has a kind (`send`, `publish`, `request`, `reply`), two endpoint-member paths, a named top-level system connector, an optional earlier-step predecessor and a unique label. Case endpoints must be visible system-level component ports; internal component interactions require a separate scenario in a system assembled at that abstraction level. This limitation keeps case paths subject to the same hierarchy rule as structural connectors.

Both endpoints MUST identify the same member of the connector's interface. Sender-receiver steps use data, publish steps use events, and request/reply steps use operation signatures. A request follows client → server or requires → provides orientation; a reply reverses the matching request. A service publish goes from provides → requires; ordinary pub/sub follows publisher → subscriber.

`after label` imposes a happens-before dependency on an earlier declared step. No predecessor means no ordering is imposed relative to other steps. Source order is only a stable display tie-break, not an implicit total order. Thus unrelated notifications can remain concurrent in SV. A reply MUST have exactly one unanswered matching request among its transitive `after` ancestors, with reversed endpoints and the same connector/member. Zero or multiple candidates is an error (SW083); each request has at most one reply. A request may remain unanswered in a partial trace. A prior reply closes a candidate only when that reply is itself an ancestor of the current step. A reply on an independent causal branch cannot resolve ambiguity merely by appearing earlier in the file. Source order never chooses between ambiguous correlations. Realization by operation bodies is a separate analysis claim requiring a witness; route-valid cases remain unverified until that evidence exists.

Cases describe representative traces, not exhaustive behavior. They do not execute a second program, introduce independent event definitions, or loosen operation guards. A complete behavioral verification must check traces against operation semantics and environmental assumptions.

4.4 View declarations

A view declaration names a view kind and a dot-separated subject in its own package. GV and GRV may reference any resolvable named semantic declaration or the owning package, except another view; IV requires a system or component; CV references a case or system, SV references a case or system (for case selection), and FV references an operation. A view of an instance resolves through its definition while retaining the occurrence path and identity.

View declarations carry no executable behavior. Layout, color, filters and collapsed regions are presentation data. The full graphical contract defines representation and source-edit invariants.

4.5 Static validity rule groups

The following groups collectively define semantic validity. Diagnostic numbers identify the present implementation's related checks, not a license to omit unimplemented clauses.

Group	Required checks	Prototype family
Names	version, uniqueness, imports, reference target kind, ambiguity, lexical scopes	SW001-SW004, linker
Contracts	interface kind/member compatibility, role compatibility, exact interface identity	SW010-SW013, SW031-SW032
Values	nominal typing, arity, field completeness, enum membership, array shape, literal ranges, purity, no recursive value containment	SW020-SW028
Architecture	acyclic composition, immediate-child boundaries, one delegation, no duplicate links/aliases, closure/cardinality	SW030-SW035
Time/QoS	positive/bounded values, explicit policy, sampled buffering, WCET/deadline/period, realtime completeness	SW040-SW045
Behavior	local-only calls/data, outgoing provenance, signatures, unique handlers, return coverage, terminating error paths, bounded loops, no recursion	SW050-SW059
States	legal stateful signature, final-state rules, priority, reachability, completion-cycle restrictions	SW060-SW064
Allocation	existing instances and nodes, no duplicate exact deployments	SW070
Presentation/cases	subject kind, connector/member paths, happens-before and uniquely correlated replies	SW080-SW083

4.6 Conformance levels

Syntax conformance (S): the complete concrete grammar, exact version, tokenization and AST construction. Recovery during editing must retain syntax diagnostics; an incomplete AST is not a valid model.

Static conformance (V): S plus every static validity rule in this specification, classified explicitly in chapter 8. A partial validator MUST enumerate checks that it lacks. It must not hide unresolved links or a validator crash as success.

KerML mapping conformance (K): V plus chapter 5's normalized elements and preserved SWAML semantic constraints, with a tested mapping against KerML 1.0 and its selected libraries. Export must retain a reverse trace map and stable identity. K does not automatically imply SWAML execution.

Execution conformance (E): V plus the operational, correlation, queue, timing-observation, failure and concurrency semantics. Target binding assumptions must be explicit. A simulator may abstract physical time while still reporting which timing obligations it does not check.

Graphical conformance (G): S and the graphical specification's projection and edit rules. G may present invalid models with diagnostics, but must not invent valid connectivity from layout. FV projects operation-owned statement flow and state behavior.

A tool claiming `SWAML 0.1 S, partial V` accurately describes this repository's current core. It must not advertise K, E or G until those implementations and checks exist. Future versions must use a new language header when breaking syntax or semantics; migration tooling must not change the meaning of models silently.

NORMATIVE / LANGUAGE CONTRACT

5. KerML semantic mapping

Normative SWAML mapping contract, with implementation status stated explicitly. Part of the SWAML 0.1 specification.

5.1 Foundation and boundary

SWAML defines software-specific specializations and constraints over a normalized KerML model. Its concrete syntax is independent: a `.swaml` file is not automatically valid `.kerml` text. The Langium AST is an authoring representation, not itself the KerML abstract syntax.

The selected foundation is OMG KerML 1.0. KerML supplies structural classifiers/features, behavior classifiers/steps, connectors, successions and expressions. The relevant normative kernel concepts are in clauses 7.3-7.4 and their abstract-syntax/semantic counterparts. SWAML's mapping below is this project's proposed specialization contract, not an OMG-approved extension. KerML specification.

The mapping inventory is a machine-readable design contract. This release does not implement or validate a KerML exporter, does not bundle a kernel semantic library, and does not claim K conformance. Implementers must test the normalized representation against the selected KerML metamodel and library before making that claim.

5.2 Semantic specializations

SWAML element	Proposed KerML base/representation	SWAML restriction or additional meaning
Package	Package	flat authoring namespace
Component definition	Structure	only composition, interfaces and connectors
Unit definition	Structure	encapsulated state and local executable features
Component/unit instance	composite Feature typed by its Structure	exactly-one multiplicity and acyclic composition
Value datatype / enum	DataType	finite value semantics; enum domain restriction
Interface contract	Classifier	nominal version, legal members and communication mode
Port	Feature typed by contract Classifier	direction/role projection and boundary ownership
Operation signature	Behavior plus typed parameter Features	no implementation; copied request/result contract
Operation implementation	Behavior and an owning-unit executable Step typed by it	direct ownership, activation, serialization and timing
Pure function	Function	parameter-only environment and no side effects
Connector	Connector between port Features	permitted interface exchange and QoS obligations
Delegation	Connector plus identity-preserving forwarding constraint	same role and contract; no extra queue
Ordered flow	contained Steps and Successions	structured branches/loops, bounded execution

SWAML element	Proposed KerML base/representation	SWAML restriction or additional meaning
Message emission	Step typed by a SWAML-library communication Behavior	source activation, route and correlation metadata
State / transition	contained behavior steps, state predicate and succession relations	SWAML state-selection and reaction semantics
Requirement / case / view	annotated named elements and trace memberships	SWAML-specific interpretation

The table establishes a mapping target, not a claim that names such as “SWAML state” already exist as native KerML metaclasses. A full exporter needs a versioned SWAML semantic library for the additional predicates, activation and communication constraints. Chapter 7 completes the information model, reference edges and lowering rules for every syntax family; the inventory covers all 80 grammar rule/type names.

5.3 Normalization algorithm

A mapper MUST perform the following in dependency order:

1. Validate the complete workspace and resolve every textual reference. Allocate stable IDs for package declarations and each named semantic element. Retain file URI and source range as provenance, not as identity.
2. Create value types and interface classifiers. Materialize every data/event/operation parameter and result as a typed feature with copied-value semantics. Preserve interface version and member order.
3. Create component/unit structures and owned instance features, with membership and typing links. Keep definition identity separate from every instance-path identity. A reused definition must not merge the states of its occurrences.
4. Create each port feature in its owning structure. Apply role-specific direction to each member projection: RPC inputs/results have opposite directions; service events run provider-to-consumer. Do not assign a single simplistic “out” bit to an entire bidirectional service contract.
5. Create operation behaviors within their unit's ownership. Add a synthetic owned executable step typed by that behavior and a unit-instance context binding. This lowering artifact gives execution a structural context; it is not a user-visible performer or a second editable allocation model.
6. Lower statement blocks to structured steps with explicit sequencing. A branch has a Boolean selection predicate and branch result control; a bounded loop has condition, body, count and failure edge. Lower expressions as typed expressions/functions. A mapper must preserve source order and short-circuit evaluation.
7. Lower each stateful operation to one state identity feature per activation, nested state-entry/exit/effect behaviors, trigger predicates, priority selection and succession relationships. Preserve finality, timer start/cancel and the run-to-completion relation. A bare succession graph without these constraints is not an equivalent mapping.
8. Lower communication statements to message occurrences linked to the emitting activation, sending port, interface member, connector/delegation route and destination port. Requests additionally carry correlation, timeout and response/error continuation. An event cannot become an unowned signal merely because a kernel target permits it.
9. Materialize queues, policy constraints, timing/deployment annotations and case happens-before relations in the SWAML semantic library vocabulary. Keep queue identity per connector/direction/member.

10. Produce a reverse mapping from every generated element to its authoring element, including synthetic lowering elements. Validate kernel typing/membership constraints and SWAML constraints separately. Report failures at the source element, not only at generated IDs.

5.4 Required extension predicates

The semantic library MUST define, or a mapper MUST carry as explicit constrained metadata, at least:

- `ownsExecutable(unit, operation)` and instance-local `contextOf(activation)`;
- `contractOf(port)`, `roleOf(port)`, `memberOf(contract, member)`;
- `route(sourcePort, destinationPort, connectorPath)`;
- `emits(activation, port, member, copiedPayload)` and `handles(operation, port, member)`;
- `correlates(request, response)`, `expiresAt(request)` and `lateResponseDiscarded(response)`;
- `queueOf(connector, direction, member)`, capacity, overflow, ordering and delivery policy;
- `currentState(activation)`, `enabled(transition, stimulus)`, priority and final-state completion;
- `wcet(operation)`, `deadline(operation)`, `period(operation)` and instance allocation;
- `happensBefore(caseStepA, caseStepB)` and `tracesTo(case, requirement)`.

These predicates are part of SWAML meaning even if an exchange format stores them as metadata. A consumer lacking their interpretation MUST report a partial import and MUST NOT execute a “best guess.” A generic KerML consumer can preserve structural content while lacking SWAML runtime semantics.

5.5 Interchange and round-trip obligations

A future exporter SHOULD use the standard KerML interchange representation supported by its chosen backend, with a declared SWAML semantic library version. No new unregistered MIME type, XMI metamodel URI, or fake OMG namespace is assigned by this document. SWAML library identifiers must use a project-controlled namespace when published.

Round-trip tests MUST compare semantic identities, typing, owned instances, routes, triggers, guards, flow ordering, queue policies, timing and trace dependencies. Byte-for-byte textual equality is not required. Comments, formatting and diagram layouts are separate preservation requirements for an authoring tool.

Dropping a guard, converting a queued event to a sampled value, merging per-subscriber queues, or omitting a request's response path is a semantic loss and must be reported as an error for K conformance. The mapping must not infer a performer allocation that changes the operation's direct owning unit.

NORMATIVE / LANGUAGE CONTRACT

6. Operational contract and target bindings

Normative chapter of the SWAML 0.1 specification. This chapter makes the allowed traces precise; it does not prescribe an implementation thread model. The executable backend remains future work.

6.1 Configuration and observable occurrences

A running configuration consists of: the selected system instance tree; private variable stores per unit instance; live activation frames and their local call stacks; one serial lock per unit; the state and timer generation of each live stateful operation; message queues keyed by connector occurrence, direction and member; pending request correlations; released lifecycle activations; and a monotonic logical time.

Each message has a logical identity, original emission time, producing activation identity, typed copied payload, source/destination port occurrences, interface member, connector occurrence and delegation route. A retry keeps the logical identity and original timestamp. A transport attempt has a separate identity. An RPC response additionally references its request correlation. Occurrence identities MUST be unique within a run and cannot be declaration IDs reused for every emission.

The minimum observable trace alphabet is initialization, release, dispatch, statement completion, emission, admission, drop, delivery, request completion, timeout, state entry/exit, activation completion/failure, and contract violation. Each record includes logical time, occurrence identity and relevant source identities. A backend may omit detailed statement traces in normal operation, but MUST provide sufficient checking instrumentation to diagnose violated obligations. Trace records and diagnostics are observations, not implicit interface messages.

A step of execution is either a local statement/expression step, communication admission/delivery, dispatch, reaction microstep, or time advancement. Independent units may step in either order. No text-file order between units creates a happens-before edge. Elapsed physical execution time must be represented when checking deadlines; treating each statement as zero-time cannot establish production timing compliance.

6.2 Initialization and release

Initialization has a barrier with the following phases:

1. Expand the acyclic instance tree, establish effective deployments, create private stores from pure initializers, resolve all routes and install all queues/handlers.
2. Install each stateful operation's initial state identity and arm its initialization reaction. It is not yet ready to consume a stimulus. Register all ordinary startup roots.
3. Establish one system epoch `t0`. Release every startup/initialization activation at `t0`; activate periodic releases at `t0 + k * period`, with k starting at 1. All ports may now admit traffic.

The scheduler may interleave startup roots across units and within a unit subject to its lock. Messages arriving before a machine finishes initial entry remain in their connector queue; they are not discarded merely because initial entry is pending. Timers start when entry finishes, not when the initialization reaction was released. A startup operation may request a service whose unit's startup root has not run: handlers and initialized data already exist. If an application needs an additional readiness protocol, it MUST model that protocol through an interface.

Each startup root is released exactly once per system run. The core defines no automatic component restart. A deployment implementation restarting a failed unit starts a new run epoch for the affected binding

and MUST report its reset, queue-retention and correlation policy; it cannot imply uninterrupted E conformance for a reset it has not modeled.

Periodic releases are independent of earlier completions. Serial dispatch does not merge or silently skip them. Manual activations are parameterless local harness releases. Input-triggered work remains in the input queue until dispatch is possible; it is not copied into an unbounded second activation queue. Admission of an RPC response resumes its suspended frame without acquiring another unit lock.

6.3 Dispatch, backlog and snapshots

A serial root acquires its unit's lock at dispatch and releases it on completion/failure. Local invokes execute on the root's stack and inherit its concurrency mode and snapshot, regardless of a helper's own timing declaration. A reentrant root may overlap other roots, but neither it nor any reachable helper may assign unit data. Assigning its own parameters/locals is allowed.

A reentrant root captures an atomic value snapshot of all unit variables at dispatch, before its first expression. All reads by that root and its helpers use that snapshot throughout suspension and resumption. An assignment by a serial activation is atomic at the whole-value level; snapshots observe some consistent store between assignments. A machine's next reaction captures a fresh snapshot if it is reentrant. Reentrant does not allow overlapping reactions of the same machine.

Between independent ready queues and lifecycle releases, the scheduler chooses any enabled root. For a FIFO queue, dispatch preserves queue order; completion order of reentrant handlers need not match dispatch order. A target backend MUST specify fairness and scheduling policy, and MUST NOT introduce starvation while claiming eventual at-least-once delivery under continuously enabled receivers.

An executable target binding MUST bound pending start/manual/periodic activations separately from connector queues. On exceeding that bound it MUST record `ActivationOverflow` and stop the affected system execution; silently dropping or coalescing releases is not an implementation of this core. A simulator may expose an explicitly unbounded mathematical backlog, but cannot use it as evidence for a bounded production deployment. The binding also bounds simultaneous reentrant roots; when full, subsequent work remains pending rather than exceeding the limit.

WCET includes local helpers and active reaction microsteps; request wait consumes deadline budget but no active CPU budget. The root's deadline is measured from release. For input reactions release occurs when the message is ready in its destination queue. A timer reaction is released at its due time even if dispatch is later. Local helpers have no independent release; a backend checks their declared execution bounds over the invocation and also includes them in the root's bounds.

6.4 Queue offers and fanout

`send` and `publish` evaluate all arguments once and snapshot the matching active connector set. They offer a distinct per-connector logical message to **every** connector in that set, even if an earlier offer rejects. Offers are independent and have no specified interconnector order. The statement finishes after all local admission decisions. If any offer rejects, the activation fails with an admission failure; successful offers remain committed. Loss through dropOldest/dropNewest is not rejection and does not fail the sender. Atomic broadcast requires a modeled protocol.

For a given queue, offer and dispatch are atomic decisions. Capacity counts retained messages waiting for dispatch, including ones whose transport availability is pending; it excludes running handlers. Removing an old message or discarding a new one MUST be observable to a checking executor. An optional port with no connectors produces no offers and succeeds for data/events.

Condition	Data/event emission	RPC request	RPC response
No live endpoint	optional unbound succeeds; failed bound route reports transport failure	`Unavailable` if unbound; otherwise `Transport` when detected	requester gets `Transport` if detectable, otherwise timeout
Full queue with reject	producer activation fails after all fanout offers	caller executes `otherwise` with `Rejected`	requester gets `Rejected` if the binding can report it; otherwise timeout; server's completed effects remain
dropOldest/dropNewest	dropped occurrence is observable; statement can succeed	dropped request waits until timeout unless transport reports failure	dropped response leaves request waiting until timeout
Age expired	discard before dispatch	no service dispatch; caller eventually times out unless already failed	discard; no successful completion
Receiver handler fails	receiver records local failure; sender is not retrospectively failed	correlated `RemoteFailure`	not a second handler; resume request failure path

Control failures such as admission rejection use the same request route/correlation. They are transport outcomes of that interface exchange, not arbitrary model event channels. A binding unable to signal a failure cannot fabricate a successful result.

An admitted message becomes ready no later than emission plus maxLatency when such a bound is claimed. Deadline miss is a violation even if the message is later delivered; it is not itself a new delivery policy. `maxAge` eligibility is inclusive: age <= maxAge may dispatch; age > maxAge is expired. Discarded/expired messages have no eventual-delivery obligation. Without maxAge, asynchronous messages have no core expiry before connection teardown; a backend must maintain its delivery obligation or explicitly report loss of availability. Retry policy, finite memory and resource exhaustion behavior belong in the target binding, not undocumented defaults.

6.5 RPC completion and races

A request correlation has states Pending, Succeeded or Failed. Creation precedes the first request offer. Its expiry is issuance time plus timeout. At most one terminal completion may win. A response can succeed only if it is observed at the requester at a time **strictly earlier** than expiry and the correlation is Pending. At expiry timeout wins ties. Terminal state is immutable; later responses and duplicate replies are discarded and observable.

AtLeastOnce may dispatch multiple handler activations with the same logical request/correlation. Idempotency must cover storage writes and published side effects; returning the same value alone is insufficient. The first eligible response completes the caller. Retrying must never create a new caller-visible correlation or extend its timeout. A binding may deduplicate repeated requests while preserving required behavior.

The mandatory `as name` on a Void RPC labels successful completion without introducing an inhabitable value. It reserves that name in the enclosing lexical scope but it MUST NOT be read or assigned as data. A Void handler uses bare `return;`. The failure block has an implicit copied `failure: String`; an existing live value with that name is a static error. Failure blocks cannot fall through to the successful continuation.

A timeout does not retract messages already sent or cancel a server frame. Pending requests cannot be used as detached futures; each statement awaits its own outcome. Synchronous dependency cycles remain expressible but may time out. A static validator is not required to prove their absence; an execution/timing claim must account for them.

6.6 State reaction ordering

A machine has one initialization reaction and then a sequence of reactions. Each reaction includes all immediately enabled completion transitions until quiescent/final/failure. Completion selection precedes dispatch of the next external stimulus. If no completion guard holds, the machine becomes quiescent. A write by another operation does not itself awaken completion transitions; they are reconsidered after the next machine reaction, including one that consumed an unhandled message or expired a timer with no enabled transition.

On completing a state's entry, create one timer stimulus per distinct `after` duration on its outgoing transitions, tagged with the current state-entry generation. All transitions with that duration compete using guard/priority. Distinct durations produce distinct stimuli. Leaving a state cancels every timer from that generation. A queued timer from an obsolete generation is discarded. Reentering, including a self-transition, creates a new generation. Entry code time is outside the residence timer; the timer measures time since entry completion.

A due timer and a ready input are independent stimuli. Their selection order is unspecified unless the target scheduler constrains it. Transition priority selects among transitions for the **same** stimulus; it is not a task/queue priority. A consumed stimulus is not retained merely because all guards were false. Entry/exit blocks have no event payload scope. Exit and effect run before state identity changes; destination entry follows that change.

The completion microstep counter is zero at the start of a reaction and increases once per taken completion transition. Taking a 1,025th completion transition fails the reaction before executing that transition. External dispatch does not occur between those microsteps. Guards are pure expressions and cannot send, mutate, or consume another message. A guard evaluation failure fails the owning activation.

Final entry completion, bare return or failure terminates the machine, cancels all its timers and disables its dispatch. Future messages to its members become undeliverable occurrences; they are not rerouted to another operation. Other operations in the same unit may remain alive. No implicit exit behavior runs on return/failure unless it has already been reached in the specified transition phases.

6.7 Execution binding contract

Every backend claiming E MUST identify its backend version, supported SWAML version, system, model digest and target configuration. It MUST supply explicit values or a justified symbolic abstraction for:

- effective deployment of every unit; clock basis, precision and cross-node error bounds;
- scheduler, fairness, serial locking, snapshot implementation and reentrant limits;
- lifecycle backlog limits and overflow observation;
- transport, serialization, copying/isolation, queue storage and retry/deduplication policy;
- deadline/latency/age measurement points and what happens after a reported violation;
- external adapter behavior and assumptions, startup readiness, shutdown/reset and queue retention;
- resource failure handling, correlation uniqueness, observability and evidence artifacts.

Timing/resource failure policies may stop execution after recording a violation. Continuing is permitted only with the violation visible, and cannot count as a satisfying execution for the violated contract. An execution trace is conforming when all observed steps obey core transitions; satisfying a particular model additionally requires its declared assertions and contracts to hold. A runtime that correctly reports a failed assertion is not thereby a broken interpreter.

Security and target-specific bus/OS policies can strengthen this contract, but cannot add hidden model exchanges. AUTOSAR Classic task/RTE mappings, Adaptive service bindings and broker configurations must preserve the same identities, member directions and copied-value boundaries.

NORMATIVE / LANGUAGE CONTRACT

7. Normalized semantic model

Normative SWAML contract extending chapter 5. This chapter defines the complete information a mapper must preserve. It specifies SWAML normalization, not a new OMG metamodel or an implemented serializer.

7.1 Identity, ownership and source trace

Normalization consumes a resolved workspace and emits a finite declaration graph plus instance expansions for selected systems. Each semantic node has an immutable identity within that normalized artifact, a semantic kind, at most one owning node, ordered owned children where order matters, typed reference edges, and source provenance. Except for model roots, every semantic node has exactly one owner. Reference edges never imply containment.

The deterministic textual identity of a declaration is its package plus owned-name path, for example `Settings::SettingStore.update`. Anonymous syntax nodes use the owning declaration ID plus a structural slot path such as `/body/statements/2/condition`; these IDs are stable for an unchanged syntax tree, not across arbitrary edits. Synthetic elements append a reserved role segment such as `/@execution` or `/@failure`. No source identifier contains `/` or `@`, so generated roles cannot collide with source names.

An occurrence identity consists of the system declaration ID and the child-instance path. A port occurrence adds the owned port name. An operation occurrence additionally refers to its shared operation declaration and unit occurrence. Runtime activations add a run ID and unique activation ordinal. A mapper MUST NOT collapse occurrence paths merely because they share a definition. Persistent editor UUIDs are a separate identity map with explicit rename/move history; source locations and in-memory AST addresses are never persistent IDs.

A reverse trace relation maps every generated element to one or more authoring node identities and their URI/ranges. Synthetic nodes retain their source owner and generation role. No generated constraint may be untraceable. An import can have an import-declaration source location while referencing a package located in another document.

7.2 Declaration graph and links

Normalized family	Required information and constraints
Workspace/package	language version, packages, direct import edges, unique declaration namespaces
Record/enum	nominal identity, ordered fields or ordered unique literal names, finite domain/containment
Type use	primitive or nominal target, optional array extent; never encode a type solely by a display label
Interface	kind, version, ordered data/event/RPC members, signatures, data mode and idempotency claims
Structure definition	component/unit kind, owned ports, contained instance features, private variables, owned executable declarations as applicable
Instance feature	component/unit kind, exactly one definition target, composite ownership, multiplicity one
Port feature	interface target, role, optionality and derived member directions

Normalized family	Required information and constraints
Connection/delegation	owner, endpoint feature references, explicit path provenance, interface identity, queue policy for a connection; no queue for delegation
Variable/local/parameter	declared type, ownership/scope, initializer where present, mutable copied-value semantics
Executable	signature, trigger, concurrency/timing, one body or machine, context bound to the owning unit
System/binding	component occurrence tree, execution nodes, explicit and effective deployment, service/realtime/mixed profile
Requirement/case/view	text, trace edges, scenario step graph, view kind and resolved subject with definition/occurrence distinction

All textual references MUST resolve before normalization. A `TypeRef` is a use, not a new nominal declaration. Array extents are carried as shape constraints and do not create element aliasing. Empty records have one value. Void results create no output-value feature. Parameters preserve order, independent of parameter names at a handler implementation.

A component instance expands its definition's child features recursively. Expansion preserves each connector's owner path, so two uses of the same component produce distinct connector occurrences and queues. Effective deployment is computed by the nearest explicitly deployed ancestor, including the occurrence itself. An absent ancestor yields an explicit Unallocated value, not an invented node.

7.3 Contract projection and route normalization

For every port occurrence and interface member, derive a member projection with contract/member identity and role-specific directions:

Role	Data/event projection	RPC request	RPC result/failure
sender / receiver	out / in data	absent	absent
publisher / subscriber	out / in event	absent	absent
client / server	absent	out / in	in / out
requires / provides	in / out event	out / in	in / out

Contract signatures remain shared; these projections belong to specific port occurrences. One bit on a service port cannot replace this table. Every receiving projection references exactly one handling operation declaration within its unit context. Every outgoing data/event projection references the nonempty set of syntactic emission sites. Those links do not assert reachability.

Resolve a connection by following each boundary port's unique delegation chain to a unit port. Preserve the ordered original boundary chain as provenance. The normalized route is `(connectorOccurrence, sourceLeafPort, destinationLeafPort, interface)`. Delegation aliases do not add storage, queues, dispatch, retries or observable messages. Distinct connectors resolving to the same directed leaf pair are invalid. Optionality is checked at every port along a boundary chain; one optional declaration cannot erase a required declaration.

A queue key is `(connectorOccurrence, direction, memberIdentity)` with direction relative to the connector's declared orientation. RPC responses and requests use opposite directions. Service events use the response direction but their distinct event member identities prevent queue sharing with results. Correlation state is separate from queue storage.

7.4 Executable normalization

Every operation has a behavior declaration and one synthetic executable feature owned by its unit structure. The feature is typed by that behavior and binds an activation to a unit occurrence. It MUST NOT create a user-visible performer, an independently editable implementation allocation, or permission to call it across a unit boundary.

A statement body normalizes to an entry, ordered steps, exit/failure terminals and lexical scopes. Every step records its source construct and ordered operands. The following lowerings are required:

Source construct	Normalized meaning
let / set	evaluate expression, then create/replace copied local or private value
send / publish	operand evaluation followed by per-route message offers, emission provenance and success/rejection continuation
request	issuance/correlation creation, suspended frame, timeout, typed success binding or terminating failure branch
invoke	synchronous same-unit call edge, copied arguments, borrowed root context/snapshot and caller continuation
return / fail	terminate the current frame; unwind callers on failure; correlated result/failure for RPC roots
if	evaluate Bool once, mutually exclusive selected branch and merge of control only
while	repeated condition, body, iteration counter and bound-failure edge; no implicit unbounded loop
assert	Bool evaluation and failure edge when false
function	expression tree with parameter-only environment and explicit acyclic function call edges

Expressions carry inferred type and evaluation order. Binary short-circuit nodes conditionally skip their right operand. A record value maps fields to declared identities, not input order; an array preserves value order and fixed extent; enum values retain enum and literal identity. Numeric failures and invalid index edges are part of the expression contract. A mapper MUST NOT silently substitute mathematical unbounded integers for SWAML bounded Int execution.

A machine is a behavior region owned by one operation. It normalizes to state identities, initial reference, entry/exit/effect step groups, trigger matchers, pure guards, priorities, finality, current-state feature, timer-generation features and the chapter 6 dispatch/reaction rules. A transition is not merely an unconstrained edge: its priority selection, source-state condition, payload environment and ordered exit/effect/update/entry phases are required semantics.

7.5 Kernel representation and SWAML library responsibilities

The `kernel` field of the mapping inventory lists proposed KerML bases; the `semantic` field names the SWAML meaning to preserve. Empty kernel lists mean a lexical/grouping node contributes to its owner's representation, not that its meaning may be discarded. The inventory covers every authored grammar rule and type alias; punctuation and reserved keywords are covered by their productions.

Kernel ownership, membership, typing, parameter, feature-value and reference relationships MUST be instantiated according to the selected KerML backend. The inventory is not a recipe to instantiate abstract metaclasses blindly. A backend must choose suitable concrete subclasses for abstract bases such as

Expression and validate all resulting relationships. Source grouping rules do not generate duplicate nodes for the same AST element.

The SWAML library supplies typed semantics for activation/context, copied values, versioned port contracts, queue offers, request correlation, failure outcomes, bounded flow and machine reactions. A communication lowering uses a Step typed by the library's send/publish/request behavior, with typed features for its ports, member and payload. It MUST NOT serialize a made-up kernel class called `SWAML-library-message-interaction`. The kernel Step is the base; the library behavior is the additional meaning.

Requirements, timing, QoS, deployment, case and view annotations use project-owned metadata definitions with typed fields and explicit references. Cases additionally carry a scenario behavior/step graph and happens-before edges; they do not become an independently executing program. A view's subject uses semantic identity and occurrence context; geometry/layout stays in a separate sidecar.

A K-conforming artifact MUST include or resolvably identify its exact SWAML library version and declare which constraints are checked. Carrying uninterpreted metadata is preservation, not proof that a generic kernel tool understands or executes it. The project has not yet selected/published a library namespace or implemented this backend; these obligations define its acceptance criteria.

7.6 Case and view normalization

Normalize each case label to a distinct step identity, resolve its route/member and orientation, and translate `after` to an explicit happens-before edge. The transitive closure determines causal ancestry. A reply links to its unique unanswered request ancestor with reversed endpoints and the same connector/member. A request is already answered for candidate selection only if its reply is also a causal ancestor; an independent branch cannot settle the choice. Ambiguous candidates and multiple replies to one request are invalid. Source position contributes display order only. An unanswered request is allowed in a partial trace; no synthetic reply is inserted.

A case realization claim records a witness execution and assumptions, including payload values absent from the diagram, dispatch choices and operation paths. Mere route validity does not supply that witness. A graphical renderer can display an unverified case but must not show it as proven.

Normalize views to `(kind, subjectDeclaration, occurrenceContext?)`. GV/GRV can inspect any resolvable named semantic declaration or the owning package, except another view. IV requires a system or component; CV/SV require a case or system; FV requires an operation. A package name colliding with a top-level declaration makes that root ambiguous and is rejected for view lookup. The dot path begins at a top-level declaration in the owning package or is the package's name alone. Descents follow component/unit instances, their structural members and interface members exposed through ports. State-level navigation is Browser/FV selection within an operation, not a second view subject syntax.

7.7 Equivalence and completeness

Two normalized artifacts are equivalent when an identity-respecting graph correspondence preserves ownership, types, value shapes, interface versions, instance contexts, routes, evaluation order, triggers, state selection, policies, deployments, case dependencies/correlations and view subjects. Whitespace, comments, source offsets and diagram geometry are excluded. Requirements text and failure reasons remain observable documentation/diagnostic values and must be preserved.

A round trip may rename anonymous lowering IDs while retaining their owner/role trace. It may not remove guarded branches, reorder effects, merge queues, weaken timing contracts, convert copied payloads into remote references, or drop SWAML metadata because a receiver lacks support. Unsupported meaning requires an explicit partial-import result. This information-preservation contract is complete for the closed 0.1 syntax; implementing and validating a KerML backend is a separate milestone.

NORMATIVE / LANGUAGE CONTRACT

8. Conformance obligations and evidence

Normative classification of the SWAML 0.1 specification. This chapter separates static language validity, runtime behavior and proof claims. It prevents acceptance by one tool from being mistaken for evidence supplied by another.

8.1 Static acceptance

Static validity is determined from the complete loaded source workspace without executing a model. The grammar plus the following obligations define the static core. The reference core implements these groups, but remains advertised as partial V pending an independent clause-by-clause conformance suite.

ID	Static obligation	Reference-core evidence
V01	Exact header; portable tokens, escapes and safe literal domains	parser, SW001/SW025; lexical audit tests
V02	Unique package/scoped names; imports exist; references resolve uniquely before kind filtering	SW002/SW004, owner-aware linker; multi-file tests
V03	Finite nominal records/enums/arrays; no Void data; complete typed constructions	SW020-SW028; values/flow and audit fixtures
V04	Pure initializer/function environments, no cross-unit value lookup, compatible operands/arguments/results	SW022-SW024; isolation and type tests
V05	Legal interface members/port roles, exact nominal matching and signature implementations	SW010/SW011/SW031/SW032/SW052
V06	Acyclic composition, correct occurrence kinds, immediate-child routes, one delegation per boundary, no duplicate aliases/routes	SW012/SW013/SW030/SW033/SW035
V07	Required-port closure through every alias; single source/provider where required	SW034/SW035; closure fixtures
V08	Every outgoing member has syntactic emission provenance; every incoming member has one handler owner	SW050/SW051/SW055
V09	Positive safe durations, capacities, array extents, loop bounds and priorities; complete QoS; sampled buffering and idempotent RPC declaration	SW020/SW040/SW041/SW043/SW059/SW062

ID	Static obligation	Reference-core evidence
V10	WCET <= deadline; periodic deadline <= period; realtime/mixed required timing/latency/age metadata	SW042/SW044/SW045
V11	Local acyclic calls; legal invoke targets; copied input parameters; no live-name shadowing	linker, SW002/SW056/SW058
V12	Value returns and request failure paths terminate; Void completion is not a value	SW020/SW022/SW023/SW053/SW057
V13	Reentrant roots do not write unit data directly or through any reachable helper	SW054; transitive-effect audit tests
V14	Machine signature, initial reference, graph reachability, final-state rules, competing priorities and no unconditional completion cycle	SW060-SW064
V15	Valid occurrence deployment/node targets; no duplicate exact allocation	SW070
V16	View subject kind, case member/route/direction, earlier predecessor and uniquely correlated reply	SW080-SW083

“Every path returns” in V12 is structural definite termination: a sequential block contains return/fail or an if with both branches definitely terminating. A loop is never sufficient by itself, because it may run zero times or hit its bound. A request failure block obeys the same rule. This rule is intentionally conservative and does not require solving guard predicates. A request's success continuation is checked independently from its failure block.

Literal array indices, including unary sign expressions, are checked statically. General expression-derived bounds and arithmetic overflow are execution checks; a validator may additionally reject a proven inevitable failure with an explained analysis diagnostic. Syntactically present but unreachable emission still satisfies provenance; proving that an interface is useful is a stronger claim.

Imported declarations and string-valued semantic paths MUST be revalidated after a dependency change, addition or deletion. Correctness includes negative lookups: a newly added declaration can make an existing short reference ambiguous. A conservative implementation may rebuild all loaded language documents. Source recovery may suspend semantic checks on a syntactically invalid document, but syntax errors and unresolved references remain visible and the document cannot pass validation.

8.2 Runtime obligations

ID	Obligation	Required evidence for E
E01	Distinct instance stores, copied payloads and activation frames	isolation traces for reused definitions and transports

ID	Obligation	Required evidence for E
E02	Startup barrier, release epochs, serial lock, snapshots and bounded activation resources	scheduler traces including suspended RPCs and reentrant readers
E03	Per-route queues, sampled replacement, overflow, ordering, age and delivery policies	adversarial queue/duplicate/drop tests
E04	Unique correlations, response route, timeout race, failure outcome and late-response suppression	concurrent request/timeout/reply traces
E05	Structured statement order, bounded loops, failure unwinding and numeric/index checks	reference evaluator cases and failure traces
E06	State reaction order, timer generations, priority, completion bound and termination	competing input/timer and self-transition traces
E07	Timing observation and explicit deployment/resource assumptions	target binding plus measured/proved scheduling evidence

The runtime must detect dynamic out-of-bounds access, non-finite arithmetic, integer overflow, division by zero and loop/microstep-limit violations. V does not have to prove that these failures never occur. A static accepted model can describe a failing execution. Likewise V checks the idempotent declaration where required; a deployment claim that it is true requires implementation evidence.

8.3 Analysis claims

A **case realization** claim requires at least one operation-level witness trace consistent with all case dependencies, route/member identities and model assumptions. A route-valid case is only a candidate until such evidence exists. Invalid correlation is statically rejectable; guard feasibility is not implied by correlation validity.

A **requirement satisfied** claim links named requirements to cases plus test/analysis/review evidence; `satisfies` syntax alone denotes intent. A **schedulable** claim identifies the target scheduler, release assumptions, execution costs, transport/queue bounds and proof or measurement envelope. A **deterministic** claim accounts for independent stimulus ordering and reentrant completion. A **deadlock-free** claim accounts for synchronous request dependency cycles and retained serial locks.

These analyses may be conservative, bounded or incomplete, but MUST state assumptions and limitations. Failure to find a witness is not automatically proof that no witness exists. Runtime snapshot realizability is a backend obligation, not something syntax validation can establish.

8.4 Kernel, editor and graphical acceptance

K requires the complete normalized model, an implemented versioned semantic library/exporter, kernel validation and round-trip evidence. The production inventory checks coverage of the mapping design; it does not validate generated KerML because no such exporter exists yet.

G requires Browser and all six view projections, instance-aware identity, accessible notation, validated source edits, layout separation and atomic undo/redo. An LSP with generated TextMate tokens is an editor foundation, not G. Full IDE refactoring additionally requires path-aware rename/reference/navigation and persistent identity handling.

A tool's conformance report MUST identify language version, implementation version, levels claimed, rule groups checked, unsupported capabilities and evidence references. It MUST NOT promote S or partial V to K/E/G by implication. Implementation status is this repository's report.

8.5 Scope and future evolution

Completeness of this draft means every 0.1 construct has syntax, ownership, reference/type rules, static constraints, operational or descriptive meaning, a normalization contract and a conformance category. It does not mean every future software architecture concern is a built-in keyword or that all backends are implemented.

The finite boundary remains explicit: no inheritance, generics, reference-valued payloads, dynamic object creation/subscriptions/discovery, unrestricted fork/join, hierarchical/orthogonal state machines or arbitrary host-language code. Model adapters and protocols can express supported interactions; a new construct needs a versioned proposal with the same obligations. Draft clarification fixes documented here keep the `0.1` header; after language stabilization, changed valid-model meaning requires versioned migration.

NORMATIVE / LANGUAGE CONTRACT

SWAML graphical notation, version 0.1

This is the normative notation and interaction contract for a future workbench. The current repository defines the model, grammar and language server; it does not claim to ship these diagram editors.

1. Views and navigation

SWAML defines the following view families. GRV is Grid View.

ID	Name	Primary subject	Contents
Browser	Model Browser	workspace / selected subtree	ownership tree, definitions/usages, errors, references
GV	General View	package, system, component, unit or declaration	owned elements, types, properties and relations
IV	Interconnect View	system or component occurrence/definition	immediate children, exposed ports, connectors, delegation
CV	Case View	system or case	cases, requirements and intended coverage
SV	Sequence View	case, or case selected from a system	interface exchanges and happens-before dependencies
GRV	Grid View	package or named semantic declaration	model inventory, interfaces, operations, timing, allocation or traces
FV	Flow View	operation	statement flow or states/transitions owned by that operation

GV/GRV subjects exclude other views. IV accepts only a component or system; a unit is inspected through GV/FV or its containing IV. A package subject is its own name alone. These categories are validated.

FV projects an operation's statement flow or owned states. Label grouping frames with their component, unit or operation paths.

The Browser shows packages → definitions/system instances → child occurrences → ports, variables, functions and operations → flow statements or states/transitions. A definition and a usage MUST have distinct badges. Opening a usage preserves its instance path even when the editor displays members inherited from its definition.

Selection updates Properties; **Open declaration** selects the shared declaration. **Diagram** opens the most appropriate view for the selected semantic element, and **Up** goes to its owning element. These commands are requirements for the future host, not implemented commands in this package.

2. Visual alphabet

Element	Shape and required labels
System	framed rectangle, «system», name, profile
Component	rectangle with double-tab component marker, name/type; children may be nested
Unit	plain rectangle with «unit» and name/type
Definition	«component def» / «unit def»; no occurrence underline
Instance	name : Type, occurrence badge; unique instance path in tooltip
Port	small boundary square labeled name : Interface @ version and role abbreviation
Interface	contract rectangle with kind and member signatures; optional lollipop/socket glyph supplements, never replaces, text
Function	rounded rectangle with ƒ, signature and purity badge
Operation	rounded rectangle with op, signature, trigger and optional timing
State	rounded rectangle with state, name and optional entry/exit compartments
Initial / final	filled circle / circle with inner filled circle, plus accessible labels
Requirement	rectangle with «requirement», name, text or synopsis
Case	rounded case node with name; links to requirements are dashed and labeled satisfies (claimed)
Execution node	framed box with node name and mcu/process/container badge

A unit is not shown as a UML actor or a performer. Physical quantities, devices and hardware drawings may appear as named external software-facing adapters, not as geometry elements. Color MAY distinguish types but MUST NOT carry meaning without shape/text redundancy.

Port role labels are S, R, C, Svr, Pub, Sub, Prov, and Req. The full role appears in Properties and accessible text. An optional unconnected port is shown with a hollow center and optional, never hidden by default.

3. Lines and direction

Relation	Line
Sender-receiver	solid arrow from sender to receiver; «data» and interface label
Publish-subscribe	solid arrow with event marker from publisher to subscriber; one branch per connector
Client-server / combined service	solid connector from client/required to server/provided, labeled «rpc» or «service»; opposite event/reply direction indicated in member inspection
Delegation	solid elbow from boundary port to child port, labeled delegate; roles remain unchanged
Definition typing	dashed open-triangle reference labeled type; not runtime communication
Deployment	dashed arrow labeled deploy to execution node

Relation	Line
State transition	arrow labeled `trigger [guard] / effect`, with priority when set
Flow edge	directed line for local control succession; conditional branches labeled true/false
Requirement trace	dashed arrow labeled `satisfies (claimed)`

Every communication line MUST terminate at a visible or explicitly elided port, never at an operation box that conceals a cross-unit bypass. An elided path must expose the full route on inspection. A connector label includes QoS at expanded detail. Invalid connectors are shown with a diagnostic badge and text; they are not restyled into apparently valid ones.

This Mermaid figure illustrates topology only; a production IV uses the boundary-square port glyphs above:

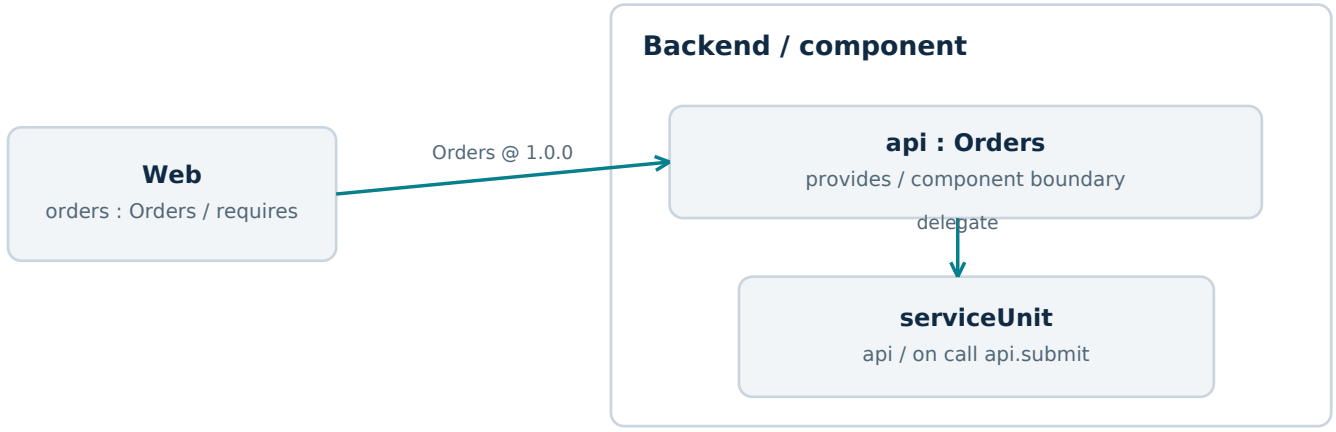


Figure 1. Delegated service contract and direct operation ownership.

The handler association inside the unit is not a second communication connector. Only the external `req-bp` line denotes an intercomponent exchange.

4. Flow View

FV has two projections under one view ID:

- **Statement body:** start, ordered statement nodes, decisions, bounded-loop back edges, request success/failure branches, local invoke nodes and end/failure terminals. A send/publish/request node carries its exact `port.member` path. Data values are labels or optional local edges, not global event nodes.
- **Stateful body:** initial state, state nodes, entry/exit compartments, transitions, timer/event labels and final states. State rectangles remain inside the owning operation frame. A state is not allocated later to a performer.



Figure 2. Request flow with explicit success and failure continuations.

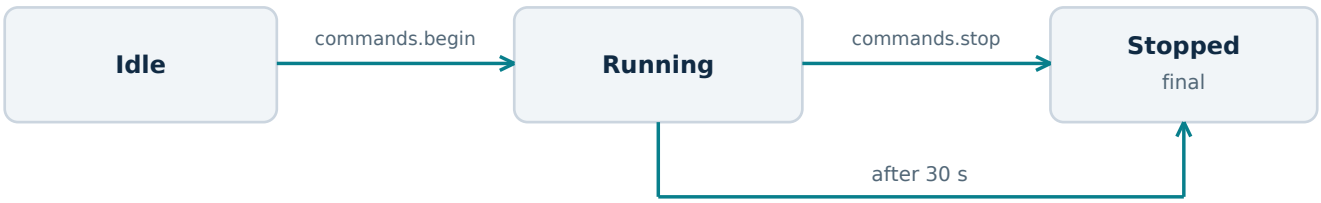


Figure 3. State behavior owned by one operation.

The state diagram is the FV of `Device.lifecycle`. In a full renderer, the frame and port-trigger labels link back to that exact operation and its interface contract.

5. Sequence and case views

SV lifelines represent component or unit **instances**, with the visible port path on each message. A request has a solid arrow and correlation label; its reply is dashed with the same correlation. Data/events use asynchronous arrows. Provider events on a combined service go in the reverse direction to client requests.

An SV renderer MUST preserve `after` dependencies and show unrelated steps as unordered/concurrent rather than inventing semantic ordering from vertical placement. It MAY choose a stable topological arrangement for readability, but must mark independent groups and expose the dependency relation. A case cannot create an operation, free event, implicit connector or arbitrary actor bypass merely by drawing a message.

CV shows the system's named cases and explicit requirement links. A filled verification checkmark requires linked evidence from a verification tool; a `satisfies` assertion alone is shown as claimed coverage.

6. Grid View

GRV is a table projection, not geometry. Available grids SHOULD include:

Grid	Minimum columns
Inventory	name, kind, owning path, definition/usage, diagnostic count
Interfaces	interface/version, member, parameters/result, port roles, emitter/handler
Connections	source, target, interface, delivery, capacity, overflow, latency, age
Operations	unit path, signature, trigger, WCET, deadline, period, concurrency
Deployment	instance, effective node, explicit/inherited allocation
Trace	requirement, case, evidence status, source location

Sorting and filtering do not change semantic order. Editing a value performs a validated source edit with undo; computed effective values must reveal the declaration that owns them.

7. Synchronization, layout and editor safety

Text and diagrams project one semantic model. A layout file stores view ID, subject identity, node positions, edge bends, collapsed regions, filters and styling only. It MUST NOT store a competing definition of ports, triggers, interfaces or connections. Source remains authoritative for semantics.

A graphical mutation MUST resolve selected semantic IDs, prepare a source edit, parse/link/validate the proposed result, report new errors, apply the edit atomically, and preserve undo. A tool may intentionally apply an incomplete editing step only if it remains visibly invalid. Moving a node changes layout only; reparenting an operation changes semantics and must validate references and interface access.

Deleting a port must reveal impacted connectors, statements and handlers. Renaming must update every typed cross-reference and path string, including cases and view subjects. Diagram nodes need stable IDs and source ranges; generated AST object identity is not stable across reparses.

The Browser, property panes and diagrams MUST support keyboard navigation and text alternatives. Error states need icons and labels in addition to color. Large models may elide details, but hidden exchanges must remain discoverable through the Browser and interface grid.

NORMATIVE APPENDIX

A. Complete concrete grammar

Authoritative Langium grammar, reproduced in full. Line wraps are typographic continuations; only numbered rows begin original source lines.

LANGIUM GRAMMAR

```
grammar Swaml

entry Model: 'swaml' version=STRING ';' packages+=Package*;
Package: 'package' name=ID '{' (imports+=Import | elements+=Element)* '}';
Import: 'import' namespace=QualifiedName '::' '*' ';';
Element: DataType | EnumType | Interface | Component | Unit | System | Requirement | Scenario | View;
DataType: 'datatype' name=ID '{' fields+=Parameter* '}';
EnumType: 'enum' name=ID '{' literals+=ID (',' literals+=ID)* '}';
Parameter: name=ID ':' type=TypeRef ';';
TypeRef: (primitive=('Bool'|'Int'|'Real'|'String'|'Void') | type=[ValueType:QualifiedName]) ('['
size=INT ']')?;
type ValueType = DataType | EnumType;
Interface: 'interface' kind=('senderReceiver'|'clientServer'|'publishSubscribe'|'service') name=ID
'version' version=STRING '{' members+=InterfaceMember* '}';
InterfaceMember: DataMember | EventMember | Signature;
DataMember: 'data' name=ID ':' type=TypeRef mode=('sampled'|'queued') ';';
EventMember: 'event' name=ID '(' (parameters+=Argument (',' parameters+=Argument)*)? ')' ';';
Signature: 'operation' name=ID '(' (parameters+=Argument (',' parameters+=Argument)*)? ')' ':'
result=TypeRef (idempotent?='idempotent')? ';';
Argument: name=ID ':' type=TypeRef;
Component: 'component' 'def' name=ID '{' members+=ComponentMember* '}';
ComponentMember: Port | Instance | Connect | Delegate;
Unit: 'unit' 'def' name=ID '{' members+=UnitMember* '}';
UnitMember: Port | Variable | Operation | FunctionDef;
type Definition = Component | Unit;
Instance: kind=('component'|'unit') name=ID ':' definition=[Definition:QualifiedName] ';';
Port: 'port' name=ID ':' interface=[Interface:QualifiedName]
role=('sender'|'receiver'|'client'|'server'|'publisher'|'subscriber'|'provides'|'requires')
(optional?='optional')? ';';
Variable: 'var' name=ID ':' type=TypeRef '=' value=Expression ';';
System: 'system' name=ID profile=('realtime'|'service'|'mixed') '{' members+=SystemMember* '}';
SystemMember: Instance | Connect | Node | Deployment;
Node: 'node' name=ID kind=('mcu'|'process'|'container') ';';
Deployment: 'deploy' target=Path 'on' node=[Node:ID] ';';
Connect: 'connect' name=ID ':' source=Path '->' target=Path qos=Qos? ';';
Delegate: 'delegate' outer=[Port:ID] '->' target=Path ';';
Qos: 'qos' '{' 'delivery' delivery=('bestEffort'|'atMostOnce'|'atLeastOnce') ';' 'ordering'
ordering=('fifo'|'unordered') ';' 'capacity' capacity=INT ';' 'overflow'
overflow=('dropOldest'|'dropNewest'|'reject') ';' ('maxLatency' maxLatency=Duration ';')? ('maxAge'
maxAge=Duration ';')? '}';
Operation: 'operation' name=ID '(' (parameters+=Argument (',' parameters+=Argument)*)? ')' ':'
result=TypeRef trigger=Trigger? timing=Timing? (body=Block | machine=Machine);
Trigger: 'on' (kind=('start'|'manual') | kind='every' period=Duration | kind=('receive'|'call')
endpoint=Path);
Timing: 'timing' '{' 'wcet' wcet=Duration ';' 'deadline' deadline=Duration ';' 'concurrency'
concurrency=('serial'|'reentrant') ';' '}';
FunctionDef: 'function' name=ID '(' (parameters+=Argument (',' parameters+=Argument)*)? ')' ':'
result=TypeRef '=' value=Expression ';';
Block: '{' statements+=Statement* '}';
Statement: Let | Assign | Send | Request | Invoke | Return | If | While | Fail | Assert;
Let: 'let' name=ID ':' type=TypeRef '=' value=Expression ';';
Assign: 'set' target=ID '=' value=Expression ';';
Send: kind=('send'|'publish') endpoint=Path '(' (args+=Expression (',' args+=Expression)*)? ')' ';';
Request: 'request' endpoint=Path '(' (args+=Expression (',' args+=Expression)*)? ')' 'as' name=ID
'within' timeout=Duration 'otherwise' failure=Block ';';
Invoke: 'invoke' operation=[Operation:ID] '(' (args+=Expression (',' args+=Expression)*)? ')' ';';
Return: 'return' value=Expression? ';';
If: 'if' '(' condition=Expression ')' then=Block ('else' otherwise=Block)?;
```

LANGIUM GRAMMAR / CONTINUED

```
While: 'while' '(' condition=Expression ')' 'bound' bound=INT body=Block;
Fail: 'fail' reason=STRING ';';
Assert: 'assert' condition=Expression ';';
Machine: 'machine' '{' 'initial' initial=[State:ID] ';' states+=State+ transitions+=Transition* '}';
State: 'state' name=ID (final?='final')? '{' ('entry' entry=Block)? ('exit' exit=Block)? '}';
Transition: 'transition' name=ID ':' from=[State:ID] '->' to=[State:ID] 'on' (kind='completion' |
kind='after' delay=Duration | kind='receive' endpoint=Path) ('when' '(' guard=Expression ')')?
('priority' priority=INT)? effect=Block;
Requirement: 'requirement' name=ID text=STRING ';';
Scenario: 'case' name=ID 'for' system=[System:QualifiedName] ('satisfies'
requirements+=[Requirement:QualifiedName] (',' requirements+=[Requirement:QualifiedName])*)? '{'
steps+=ScenarioStep* '}';
ScenarioStep: kind=('send'|'publish'|'request'|'reply') source=Path '->' target=Path 'via' connector=ID
('after' predecessor=ID)? 'label' name=ID ';';
View: 'view' name=ID kind=('GV'|'IV'|'CV'|'SV'|'GRV'|'FV') 'of' subject=Path ';';
Duration: value=INT unit=('ns'|'us'|'ms'|'s');
Path returns string: ID ('.' ID)*;
QualifiedName returns string: ID ('::' ID)*;
type Expression = BinaryExpression | UnaryExpression | Literal | FunctionCall | ReferenceExpression |
RecordLiteral | EnumLiteral | ArrayLiteral;
Expression returns Expression: OrExpression;
OrExpression returns Expression: AndExpression ({infer BinaryExpression.left=current} operator='or'
right=AndExpression)*;
AndExpression returns Expression: EqualityExpression ({infer BinaryExpression.left=current}
operator='and' right=EqualityExpression)*;
EqualityExpression returns Expression: RelationalExpression ({infer BinaryExpression.left=current}
operator=('=='|'!=') right=RelationalExpression)*;
RelationalExpression returns Expression: AdditiveExpression ({infer BinaryExpression.left=current}
operator=('<'|'<='|'>'|'>=') right=AdditiveExpression)*;
AdditiveExpression returns Expression: MultiplicativeExpression ({infer BinaryExpression.left=current}
operator=('+'|'-') right=MultiplicativeExpression)*;
MultiplicativeExpression returns Expression: UnaryExpression ({infer BinaryExpression.left=current}
operator=('*'|'/'|'%') right=UnaryExpression)*;
UnaryExpression returns Expression: {infer UnaryExpression} operator=('not'|'-') operand=UnaryExpression
 | PrimaryExpression;
PrimaryExpression returns Expression: Literal | FunctionCall | ReferenceExpression | RecordLiteral |
EnumLiteral | ArrayLiteral | '(' Expression ')';
Literal: {infer Literal} (number=Numeric | text=STRING | boolean=('true'|'false'));
FunctionCall: 'compute' function=[FunctionDef:ID] '(' (args+=Expression (',' args+=Expression)*)? ')';
ReferenceExpression: path=Path ('[' index=Expression ']')?;
RecordLiteral: 'record' type=[DataType:QualifiedName] '{' fields+=FieldValue* '}';
FieldValue: name=ID '=' value=Expression ';';
EnumLiteral: 'enumValue' type=[EnumType:QualifiedName] '.' literal=ID;
ArrayLiteral: 'array' '[' (values+=Expression (',' values+=Expression)*)? ']';
Numeric returns number: NUMBER | INT;
hidden terminal WS: /\s+/;
terminal ID: /[_a-zA-Z][\w]*/;
terminal NUMBER returns number: /[0-9]+\.[0-9]+/;
terminal INT returns number: /[0-9]+/;
terminal STRING: /"([^"\\\x00-\x1F]|\\["\\nrt])*"/;
hidden terminal ML_COMMENT: /\/\*[\s\S]*?\*\//;
hidden terminal SL_COMMENT: /\/\/[^\n\r]*/;
```

NORMATIVE DESIGN INVENTORY

B. Grammar-to-semantics inventory

Every grammar rule/type name is represented. Proposed kernel bases identify the design target; they do not constitute an implemented or validated exporter. Empty bases indicate syntax normalized through its owner.

Rule / kind	Proposed kernel base	Meaning to preserve
Model element	Package	Workspace root and version; source containers remain traceable.
Package element	Package	Flat unique declaration namespace with ordered owned members.
Import element	NamespaceImport	Direct wildcard visibility edge; no transitive import or implicit copying.
Element grouping	Owner representation	Grammar union; normalize the selected concrete alternative exactly once.
DataType element	DataType	Finite nominal value domain: ordered record fields or unique enum literals.
EnumType element	DataType	Finite nominal value domain: ordered record fields or unique enum literals.
Parameter element	Feature	Typed field/input feature; preserve declared position and copied values.
TypeRef reference	FeatureTyping	Reference to primitive/nominal type, plus optional fixed-array shape.
ValueType grouping	Owner representation	Grammar union; normalize the selected concrete alternative exactly once.
Interface element	Classifier	Nominal versioned member contract; legal member set depends on kind.
InterfaceMember grouping	Owner representation	Grammar union; normalize the selected concrete alternative exactly once.
DataMember element	Feature	Typed payload contract with sampled/queued delivery interpretation.
EventMember element	Behavior	Contract-only payload signature; only port-projected operation emission can instantiate an exchange.
Signature element	Behavior	Contract-only RPC signature with copied parameters/result and idempotency assertion.
Argument element	Feature	Typed field/input feature; preserve declared position and copied values.
Component element	Structure	Component composition or unit private-state/executable boundary; distinct occurrence contexts.
ComponentMember grouping	Owner representation	Grammar union; normalize the selected concrete alternative exactly once.

Rule / kind	Proposed kernel base	Meaning to preserve
Unit element	Structure	Component composition or unit private-state/executable boundary; distinct occurrence contexts.
UnitMember grouping	Owner representation	Grammar union; normalize the selected concrete alternative exactly once.
Definition grouping	Owner representation	Grammar union; normalize the selected concrete alternative exactly once.
Instance element	Feature	Composite owned occurrence typed by a definition, multiplicity one.
Port element	Feature	Interface-typed role projection with optionality and instance-local endpoints.
Variable element	Feature, FeatureValue	Private variable with pure initializer and per-unit-occurrence storage.
System element	Structure	Closed assembly with profile, expanded instances and resolved routes.
SystemMember grouping	Owner representation	Grammar union; normalize the selected concrete alternative exactly once.
Node element	Structure	Logical execution-node declaration with mcu/process/container binding metadata.
Deployment element	MetadataFeature	Explicit occurrence-to-node allocation; nearest ancestor determines inherited allocation.
Connect element	Connector	Port route or queue-free forwarding alias; preserve hierarchy and nominal interface identity.
Delegate element	Connector	Port route or queue-free forwarding alias; preserve hierarchy and nominal interface identity.
Qos element	MetadataFeature	Per-route/member/direction queue capacity, overflow, delivery, order, latency and age constraints.
Operation element	Behavior, Step	Owned implementation plus synthetic executable feature; typed unit context and exactly one body form.
Trigger element	MetadataFeature	Local release rule or port-member handler binding; not a free event.
Timing element	MetadataFeature	WCET, elapsed deadline and concurrency contract; root and helper interpretation.
FunctionDef element	Function	Pure parameter-only expression with acyclic local call graph.
Block element	Step, Succession	Ordered steps, lexical scope, success/failure exits and source trace.
Statement grouping	Owner representation	Grammar union; normalize the selected concrete alternative exactly once.
Let element	Step, Feature	Evaluate then create/replace one copied value in the proper lexical/private environment.

Rule / kind	Proposed kernel base	Meaning to preserve
Assign element	Step, Feature	Evaluate then create/replace one copied value in the proper lexical/private environment.
Send element	Step	SWAML library emission behavior with operation/port/member provenance and independent route offers.
Request element	Step	SWAML library awaited RPC behavior with correlation, bounded timeout and exclusive success/failure.
Invoke element	Step	Same-unit synchronous call preserving caller context, lock, snapshot and continuation.
Return element	Step	Frame completion or failure unwinding; no rollback or implicit event.
If element	Step, Succession	Guarded structured control with branch exclusivity, loop count and bound failure.
While element	Step, Succession	Guarded structured control with branch exclusivity, loop count and bound failure.
Fail element	Step	Frame completion or failure unwinding; no rollback or implicit event.
Assert element	Step	Pure Boolean check with explicit activation failure continuation.
Machine element	Behavior, Feature, Step, Succession	Owned stateful body with current-state, timer generations and ordered reactions.
State element	Step	State identity, entry/exit groups and finality in one operation context.
Transition element	Step, Succession	Source/target, stimulus matcher, guard, priority and exit/effect/update/entry sequence.
Requirement element	MetadataFeature	Named text obligation; coverage claims carry no automatic proof.
Scenario element	Behavior	Descriptive partial trace tied to a system and requirement references; not a second executable program.
ScenarioStep element	Step, Succession	Port-member exchange with connector, explicit causal predecessor and inferred unique reply correlation.
View element	MetadataFeature	Projection kind and resolved declaration/occurrence subject; presentation has no runtime effects.
Duration value	LiteralInteger	Exact positive integer nanoseconds with retained authoring unit.
Path reference	Owner representation	Resolve according to reference context; preserve authoring spelling and resolved identities.
QualifiedName reference	Owner representation	Resolve according to reference context; preserve authoring spelling and resolved identities.
Expression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.

Rule / kind	Proposed kernel base	Meaning to preserve
OrExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
AndExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
EqualityExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
RelationalExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
AdditiveExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
MultiplicativeExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
UnaryExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
PrimaryExpression lowering	Expression	Typed expression tree; grouping/precedence, operand order and conditional short-circuit evaluation.
Literal value	Expression	Concrete typed literal expression: Bool, bounded Int, finite Real or portable String.
FunctionCall lowering	Expression	Typed pure local function invocation with ordered copied arguments.
ReferenceExpression lowering	Expression	Local/copied/private value lookup with record traversal and checked final array indexing.
RecordLiteral value	Expression	Nominal construction, declared field/literal identities, fixed ordered array values.
FieldValue value	Expression	Nominal construction, declared field/literal identities, fixed ordered array values.
EnumLiteral value	Expression	Nominal construction, declared field/literal identities, fixed ordered array values.
ArrayLiteral value	Expression	Nominal construction, declared field/literal identities, fixed ordered array values.
Numeric lexical	Owner representation	Unsigned decimal value; unary minus is an expression; finite/safe literal range checks.
WS lexical	Owner representation	Insignificant source trivia; retain only for authoring round trips.
ID lexical	Owner representation	Case-sensitive ASCII identifier; grammar keywords are reserved.
NUMBER lexical	Owner representation	Unsigned decimal value; unary minus is an expression; finite/safe literal range checks.
INT lexical	Owner representation	Unsigned decimal value; unary minus is an expression; finite/safe literal range checks.

Rule / kind	Proposed kernel base	Meaning to preserve
STRING lexical	Owner representation	Unicode text with only quote, backslash, n, r and t escapes; no raw controls.
ML_COMMENT lexical	Owner representation	Insignificant source trivia; retain only for authoring round trips.
SL_COMMENT lexical	Owner representation	Insignificant source trivia; retain only for authoring round trips.

INFORMATIVE / COMPLETE MODELS

C. Model examples

01 hello

SWAML / 01-hello.swaml

```
swaml "0.1";
package Hello {
    interface senderReceiver Greeting version "1.0.0" {
        data text: String queued;
    }
    unit def Speaker {
        port output: Greeting sender;
        operation greet(): Void on start {
            send output.text("Hello, SWAML");
            return;
        }
    }
    unit def Listener {
        port input: Greeting receiver;
        var last: String = "";
        operation hear(message: String): Void on receive input.text {
            set last = message;
            return;
        }
    }
    component def Conversation {
        unit speaker: Speaker;
        unit listener: Listener;
        connect greeting: speaker.output -> listener.input qos {
            delivery atMostOnce; ordering fifo; capacity 4; overflow reject;
        };
    }
    system Demo service { component conversation: Conversation; }
    view overview GV of Demo;
    view wiring IV of Conversation;
    view greetingFlow FV of Speaker.greet;
    view inventory GRV of Demo;
}
```

02 realtime controller

SWAML / 02-realtime-controller.swaml

```
swaml "0.1";
package Control {
    interface senderReceiver Samples version "1.0.0" { data temperature: Real sampled; }
    interface senderReceiver Commands version "1.0.0" { data duty: Real sampled; }
    unit def Sensor {
        port output: Samples sender;
        operation sample(): Void on every 10 ms timing { wcet 1 ms; deadline 3 ms; concurrency serial; } {
            send output.temperature(23.5);
            return;
        }
    }
    unit def Regulator {
        port input: Samples receiver;
        port output: Commands sender;
        function clamp(x: Real): Real = x / 100.0;
        operation regulate(temperature: Real): Void on receive input.temperature timing { wcet 1 ms; deadline 3 ms; concurrency serial; } {
            let duty: Real = compute clamp(temperature);
            if (duty > 0.8) { send output.duty(0.8); }
            else { send output.duty(duty); }
            return;
        }
    }
    unit def Actuator {
        port input: Commands receiver;
        var applied: Real = 0.0;
        operation apply(duty: Real): Void on receive input.duty timing { wcet 1 ms; deadline 2 ms; concurrency serial; } {
            set applied = duty;
            assert applied <= 0.8;
            return;
        }
    }
    component def Controller {
        unit sensor: Sensor;
        unit regulator: Regulator;
        unit actuator: Actuator;
        connect measured: sensor.output -> regulator.input qos {
            delivery atMostOnce; ordering fifo; capacity 1; overflow dropOldest; maxLatency 1 ms; maxAge 20 ms;
        };
        connect commanded: regulator.output -> actuator.input qos {
            delivery atMostOnce; ordering fifo; capacity 1; overflow dropOldest; maxLatency 1 ms; maxAge 20 ms;
        };
    }
    system Heating realtime {
        component controller: Controller;
        node ecu mcu;
        deploy controller on ecu;
    }
    requirement LimitedDuty "Applied duty shall be at most 0.8.";
    view overview GV of Heating;
    view wiring IV of Controller;
    view regulating FV of Regulator.regulate;
    view deployment GRV of Heating;
}
```

03 service pubsub

SWAML / 03-service-pubsub.swaml

```
swaml "0.1";
package Commerce {
    interface service Orders version "1.0.0" {
        // Submitting publishes effects, so this contract does not claim idempotency.
        operation submit(orderId: String): Bool;
        event accepted(orderId: String);
    }
    interface publishSubscribe Audit version "1.0.0" { event recorded(orderId: String); }
    unit def OrderService {
        port api: Orders provides;
        port audit: Audit publisher;
        operation submit(orderId: String): Bool on call api.submit {
            publish api.accepted(orderId);
            publish audit.recorded(orderId);
            return true;
        }
    }
    unit def Frontend {
        port orders: Orders requires;
        var lastAccepted: String = "";
        operation checkout(): Void on manual {
            request orders.submit("order-42") as accepted within 2 s otherwise { fail "checkout unavailable"; };
            assert accepted;
            return;
        }
        operation confirmation(orderId: String): Void on receive orders.accepted { set lastAccepted = orderId; return; }
    }
    unit def AuditStore {
        port input: Audit subscriber;
        var last: String = "";
        operation storeRecord(orderId: String): Void on receive input.recorded { set last = orderId; return; }
    }
    component def Backend {
        port api: Orders provides;
        port audit: Audit publisher;
        unit serviceUnit: OrderService;
        delegate api -> serviceUnit.api;
        delegate audit -> serviceUnit.audit;
    }
    component def Web {
        port orders: Orders requires;
        unit frontend: Frontend;
        delegate orders -> frontend.orders;
    }
    component def Archive {
        port input: Audit subscriber;
        unit store: AuditStore;
        delegate input -> store.input;
    }
    system Shop service {
        component web: Web;
        component backend: Backend;
        component archive: Archive;
        connect orderLink: web.orders -> backend.api qos { delivery atMostOnce; ordering fifo; capacity 32; overflow reject; };
        connect auditing: backend.audit -> archive.input qos { delivery atLeastOnce; ordering fifo; capacity 128; overflow dropOldest; };
        node edge container;
        node application container;
        node storage container;
        deploy web on edge;
        deploy backend on application;
```

SWAML / 03-service-pubsub.swaml / CONTINUED

```
        deploy archive on storage;
    }
    requirement AcceptedOrder "Accepted orders shall be published to audit subscribers.";
    case Checkout for Shop satisfies AcceptedOrder {
        request web.orders.submit -> backend.api.submit via orderLink label submitOrder;
        publish backend.api.accepted -> web.orders.accepted via orderLink after submitOrder label notification;
        publish backend.audit.recorded -> archive.input.recorded via auditing after submitOrder label auditRecord;
        reply backend.api.submit -> web.orders.submit via orderLink after notification label response;
    }
    view overview GV of Shop;
    view wiring IV of Shop;
    view cases CV of Shop;
    view checkout SV of Checkout;
    view services GRV of Shop;
    view submitFlow FV of OrderService.submit;
}
```

04 stateful operation

SWAML / 04-stateful-operation.swaml

```
swaml "0.1";
package Lifecycle {
    interface publishSubscribe Commands version "1.0.0" { event begin(); event stop(); }
    interface publishSubscribe Status version "1.0.0" { event changed(value: String); }
    unit def Operator {
        port commands: Commands publisher;
        operation startDevice(): Void on manual { publish commands.begin(); return; }
        operation stopDevice(): Void on manual { publish commands.stop(); return; }
    }
    unit def Device {
        port commands: Commands subscriber;
        port status: Status publisher;
        var cycles: Int = 0;
        operation lifecycle(): Void on start machine {
            initial Idle;
            state Idle { entry { publish status.changed("idle"); } }
            state Running { entry { set cycles = cycles + 1; publish status.changed("running"); } }
            state Stopped final { entry { publish status.changed("stopped"); } }
            transition beginRun: Idle -> Running on receive commands.begin {}
            transition stopRun: Running -> Stopped on receive commands.stop {}
            transition expire: Running -> Stopped on after 30 s { publish status.changed("timeout"); }
        }
    }
    unit def Monitor {
        port status: Status subscriber;
        var current: String = "unknown";
        operation observe(value: String): Void on receive status.changed { set current = value; return;
}
    }
    component def Assembly {
        unit operatorUnit: Operator;
        unit device: Device;
        unit monitor: Monitor;
        connect commands: operatorUnit.commands -> device.commands qos { delivery atMostOnce; ordering
fifo; capacity 8; overflow reject; };
        connect status: device.status -> monitor.status qos { delivery atMostOnce; ordering fifo;
capacity 8; overflow dropOldest; };
    }
    system Product service { component assembly: Assembly; }
    view lifecycle FV of Device.lifecycle;
    view wiring IV of Assembly;
}
```

05 values and flow

SWAML / 05-values-and-flow.swaml

```
swaml "0.1";
package Values {
    enum Health { Good, Failed }
    datatype Measurement { value: Real; health: Health; }
    interface senderReceiver Measurements version "1.0.0" { data sample: Measurement queued; }
    unit def Producer {
        port output: Measurements sender;
        function twice(x: Real): Real = x * 2.0;
        operation prepare(): Void {
            let samples: Real[3] = array [1.0, 2.0, 3.0];
            let i: Int = 0;
            while (i < 3) bound 3 {
                let measurement: Measurement = record Measurement {
                    value = compute twice(samples[i]);
                    health = enumValue Health.Good;
                };
                send output.sample(measurement);
                set i = i + 1;
            }
            return;
        }
        operation boot(): Void on start { invoke prepare(); return; }
    }
    unit def Consumer {
        port input: Measurements receiver;
        var latest: Real = 0.0;
        operation update(measurement: Measurement): Void on receive input.sample {
            if (measurement.health == enumValue Health.Good) { set latest = measurement.value; }
            return;
        }
    }
    component def Pipeline {
        unit producer: Producer;
        unit consumer: Consumer;
        connect samples: producer.output -> consumer.input qos { delivery atMostOnce; ordering fifo;
capacity 3; overflow reject; };
    }
    system Demo service { component pipeline: Pipeline; }
    view preparation FV of Producer.prepare;
}
```

06 mixed gateway

SWAML / 06-mixed-gateway.swaml

```
swaml "0.1";
package Gateway {
    interface senderReceiver Samples version "1.0.0" { data value: Real sampled; }
    interface service Telemetry version "1.0.0" {
        operation read(): Real idempotent;
        event observed(value: Real);
    }
    unit def Sensor {
        port output: Samples sender;
        operation sample(): Void on every 10 ms timing { wcet 1 ms; deadline 2 ms; concurrency serial; } {
            send output.value(21.0); return;
        }
    }
    unit def Adapter {
        port input: Samples receiver;
        port api: Telemetry provides;
        var latest: Real = 0.0;
        operation receiveSample(value: Real): Void on receive input.value timing { wcet 1 ms; deadline 3 ms; concurrency serial; } {
            set latest = value;
            publish api.observed(value);
            return;
        }
        operation readLatest(): Real on call api.read timing { wcet 1 ms; deadline 3 ms; concurrency serial; } { return latest; }
    }
    unit def Dashboard {
        port telemetry: Telemetry requires;
        var displayed: Real = 0.0;
        operation refresh(): Void on manual timing { wcet 1 ms; deadline 30 ms; concurrency serial; } {
            request telemetry.read() as measured within 25 ms otherwise { return; };
            set displayed = measured; return;
        }
        operation update(value: Real): Void on receive telemetry.observed timing { wcet 1 ms; deadline 5 ms; concurrency serial; } {
            set displayed = value; return;
        }
    }
    component def Acquisition {
        port output: Samples sender;
        unit sensor: Sensor;
        delegate output -> sensor.output;
    }
    component def Bridge {
        port input: Samples receiver;
        port api: Telemetry provides;
        unit adapter: Adapter;
        delegate input -> adapter.input;
        delegate api -> adapter.api;
    }
    component def Display {
        port telemetry: Telemetry requires;
        unit dashboard: Dashboard;
        delegate telemetry -> dashboard.telemetry;
    }
    system Product mixed {
        component acquisition: Acquisition;
        component bridge: Bridge;
        component display: Display;
        connect sensing: acquisition.output -> bridge.input qos {
            delivery atMostOnce; ordering fifo; capacity 1; overflow dropOldest; maxLatency 1 ms; maxAge 20 ms;
        };
        connect observing: display.telemetry -> bridge.api qos {
```

SWAML / 06-mixed-gateway.swaml / CONTINUED

```
            delivery atMostOnce; ordering fifo; capacity 8; overflow reject; maxLatency 2 ms;
        };
        node controller mcu;
        node edge process;
        deploy acquisition on controller;
        deploy bridge on edge;
        deploy display on edge;
    }
    view overview GV of Product;
    view connections IV of Product;
    view conversion FV of Adapter.receiveSample;
    view allocation GRV of Product;
}
```

07 client server

SWAML / 07-client-server.swaml

```
swaml "0.1";
package Rpc {
    interface clientServer Health version "1.0.0" {
        operation check(): Bool idempotent;
    }
    unit def Server {
        port health: Health server;
        operation checkHealth(): Bool on call health.check { return true; }
    }
    unit def Client {
        port health: Health client;
        var healthy: Bool = false;
        operation poll(): Void on every 1 s {
            request health.check() as statusValue within 100 ms otherwise { set healthy = false; return;
 };
            set healthy = statusValue;
            return;
        }
    }
    component def Service {
        port health: Health server;
        unit serverUnit: Server;
        delegate health -> serverUnit.health;
    }
    component def Caller {
        port health: Health client;
        unit clientUnit: Client;
        delegate health -> clientUnit.health;
    }
    system HealthCheck service {
        component caller: Caller;
        component serviceUnit: Service;
        connect healthCheck: caller.health -> serviceUnit.health qos {
            delivery atLeastOnce; ordering fifo; capacity 4; overflow reject;
        };
    }
    case Poll for HealthCheck {
        request caller.health.check -> serviceUnit.health.check via healthCheck label sent;
        reply serviceUnit.health.check -> caller.health.check via healthCheck after sent label answered;
    }
    view interaction SV of Poll;
    view requestFlow FV of Client.poll;
}
```

08 service field

SWAML / 08-service-field.swaml

```
swaml "0.1";
package Settings {
    // A remotely visible property is an explicit getter, setter and event contract.
    interface service TemperatureSetting version "1.0.0" {
        operation get(): Int idempotent;
        operation setPoint(desired: Int): Bool;
        event changed(current: Int);
    }
    unit def SettingStore {
        port api: TemperatureSetting provides;
        var currentValue: Int = 20;
        // Each reentrant query sees its dispatch-time snapshot.
        operation read(): Int on call api.get timing {
            wcet 1 ms; deadline 5 ms; concurrency reentrant;
        } { return currentValue; }
        operation update(desired: Int): Bool on call api.setPoint {
            if (desired < 10 or desired > 30) { return false; }
            set currentValue = desired;
            publish api.changed(currentValue);
            return true;
        }
    }
    unit def Panel {
        port setting: TemperatureSetting requires;
        var displayed: Int = 20;
        operation adjust(): Void on manual {
            request setting.setPoint(22) as accepted within 100 ms otherwise { fail "setting
unavailable"; };
            assert accepted;
        }
        operation refresh(): Void on manual {
            request setting.get() as observed within 100 ms otherwise { return; };
            set displayed = observed;
        }
        operation display(current: Int): Void on receive setting.changed { set displayed = current; }
    }
    component def Service {
        port api: TemperatureSetting provides;
        unit store: SettingStore;
        delegate api -> store.api;
    }
    component def Client {
        port setting: TemperatureSetting requires;
        unit panel: Panel;
        delegate setting -> panel.setting;
    }
    system Cabin service {
        component settings: Service;
        component front: Client;
        component rear: Client;
        connect frontLink: front.setting -> settings.api qos {
            delivery atMostOnce; ordering fifo; capacity 8; overflow reject;
        };
        connect rearLink: rear.setting -> settings.api qos {
            delivery atMostOnce; ordering fifo; capacity 8; overflow reject;
        };
        node controller process;
        node displays container;
        deploy settings on controller;
        deploy front on displays;
        deploy rear on displays;
    }
    requirement VisibleSetting "Both panels can observe accepted temperature setting updates.";
    case Change for Cabin satisfies VisibleSetting {
        request front.setting.setPoint -> settings.api.setPoint via frontLink label adjust;
```

SWAML / 08-service-field.swaml / CONTINUED

```
        publish settings.api.changed -> front.setting.changed via frontLink after adjust label frontNotice;
        publish settings.api.changed -> rear.setting.changed via rearLink after adjust label rearNotice;
        reply settings.api.setPoint -> front.setting.setPoint via frontLink after frontNotice label accepted;
    }
    view catalog GRV of Settings;
    view overview GV of Cabin;
    view wiring IV of Cabin;
    view coverage CV of Cabin;
    view exchanges SV of Change;
    view updateFlow FV of Cabin.settings.store.update;
}
```

INFORMATIVE / GUIDANCE AND PROJECT STATUS

SWAML 0.1 user guide

SWAML describes what software contains, how it communicates, and how operations and state evolve. The model remains independent of C++, Rust, an RTOS, a cloud runtime, or a particular network protocol.

1. Install and check a model

From the repository root:

```sh
npm ci
npm run build
node out/src/cli/main.js examples/01-hello.swaml
```

A successful result is `Checked 1 model(s): 0 error(s)`. This checks the implemented rules; it does not execute the software or prove schedulability. `npm run check` also runs the repository tests. Open the implementation matrix before interpreting a passed check as a stronger assurance claim.

A file starts with `swaml "0.1";` and contains named packages. Names are case sensitive. Keywords cannot be identifiers. Use `orderLink`, for example, rather than the reserved word `ordering`.

2. Write the contract first

Choose the communication model by the meaning of the exchange:

Need	Interface	Roles	Operation statement
A current sensor value	`senderReceiver`, `sampled` data	sender → receiver	`send p.member(value);`
Every queued reading	`senderReceiver`, `queued` data	sender → receiver	`send p.member(value);`
Request and correlated response	`clientServer`	client → server	`request p.member(...) as result within … otherwise …;`
One event to many subscribers	`publishSubscribe`	publisher → subscriber	`publish p.member(...);`
RPC plus server notifications	`service`	requires → provides	request and publish

Each connector uses one nominal interface definition. Equal-looking interfaces are not automatically interchangeable. Create an adapter unit when their meaning, payload, or version differs.

This is a complete small model; save it as `example.swaml`:

```swaml
swaml "0.1";
package First {
    interface senderReceiver Reading version "1.0.0" {
        data value: Real sampled;
    }
    unit def Source {
        port output: Reading sender;
```

SWAML / CONTINUED

```
        operation sample(): Void on start {
            send output.value(21.5);
            return;
        }
    }
    unit def Sink {
        port input: Reading receiver;
        var last: Real = 0.0;
        operation update(value: Real): Void on receive input.value {
            set last = value;
            return;
        }
    }
    component def Pair {
        unit source: Source;
        unit sink: Sink;
        connect readings: source.output -> sink.input qos {
            delivery atMostOnce; ordering fifo; capacity 1; overflow dropOldest;
        };
    }
    system Demo service { component pair: Pair; }
    view overview GV of Demo;
    view wiring IV of Pair;
    view updateFlow FV of Sink.update;
}
```

`Reading` defines the payload. Both ports use it with complementary roles. `sample` sends through its own port, and `update` declares that same port-member contract as its activation source. The connector allows the transfer. The variable `last` belongs to each `Sink` instance, not to every instance of its definition.

3. Choose component and unit boundaries

Use a **component** for a reusable architecture boundary: an embedded controller, a service backend, or a subsystem. Use a **unit** for a cohesive implementation boundary that owns operations and private data. A unit need not be a thread, file, class, or process; deployment is separate.

A component owns component/unit instances, ports and connectors. It does not own executable statements. A unit owns operations, pure functions and private variables. This keeps executable ownership unambiguous.

An operation can call a local helper with `invoke helper(...);`. The helper must be an untriggered `Void` operation in the same unit. Use `compute functionName(...)` for pure calculation results. These local calls do not need ports because they do not cross the agreed unit boundary.

Never write `otherUnit.operation()` or `otherUnit.privateValue`. The syntax and validator reject that access. To reach another unit, declare a port, choose an interface, connect it in their common parent, and use `request`, `send` or `publish`.

For a nested component, expose a boundary port and delegate it to one immediate child port. The service example demonstrates `Backend.api` delegating to `serviceUnit.api`. Delegation preserves the port's role and interface; it does not reverse them or convert data.

4. Describe operation control flow

Operations contain ordered statements: typed local declarations, assignments, assertions, conditionals, bounded loops, local calls, interface exchanges, returns and failures. Read Values and flow as a worked example.

Functions are expressions with typed parameters and results. They cannot read mutable unit data, issue requests, or emit events. Pass their inputs explicitly. Records, enums and arrays are values, so sending them does not expose a remote reference to local mutable data.

A `while` includes a maximum iteration count. Exceeding that count at runtime is a failure, not permission to silently stop the loop. This makes the bound useful to a later analysis or code generator.

Use `assert` for a model-level assumption that must hold at that point. A requirement is a named prose obligation. A case can link to requirements using `satisfies`; that trace link is not proof that the requirement is met.

5. Model a service call and its failure

Read Services and pub/sub. The service declares `submit` in its interface; the server unit implements it with `on call api.submit`. The client's `request` includes a timeout and an `otherwise` block.

A request binds its result only on success. The failure block has a local `failure: String` value and must return from or fail the current operation. This prevents subsequent statements from reading an uninitialized result. A timeout says the caller stopped waiting; it does not prove the server did nothing. Later responses to that timed-out request are ignored.

For application-specific errors, use an enum or result record in the operation's declared return type. Transport failures and application results are different concepts. Do not retry a request unless its idempotency and retry budget are established. The core rejects `atLeastOnce` RPC when the operation is not declared idempotent; proving that declaration is an implementation obligation.

6. Use publish/subscribe without a hidden event bus

An event is a member of a `publishSubscribe` or `service` interface. Only an operation can publish it, through one of its own outgoing ports. Subscribers receive it through their own typed ports. The normal handler uses `on receive input.eventName` with the event's parameter types in order.

Fan-out is multiple explicit connectors from a publisher port to subscriber ports. Each subscription has its own queue and QoS. A slow subscriber does not silently change the queue of another subscriber. Multiple publishers may connect to one subscriber; ordering is per connector, not global.

In a combined service, the connector is drawn from `requires` to `provides` for requests. Service events travel back from provider to consumer, and replies return on the originating request's correlation. Both remain members of the same declared interface.

Model a broker as a component with incoming and outgoing interfaces if broker behavior, failure, buffering or deployment matters. A connector alone represents a logical subscription, without selecting MQTT, DDS, Kafka or SOME/IP. Dynamic discovery and unsubscribe operations are reserved for future language versions.

7. Put states inside an operation

The stateful operation example owns `Idle`, `Running`, and `Stopped` inside `Device.lifecycle`. It starts once with its unit. No performer or external behavior allocation is required.

A stateful operation returns `Void`, has no parameters and uses `on start machine { … }`. It declares an initial state, states with optional entry/exit blocks, and transitions. Triggers are an incoming port member, a positive delay after entering a state, or completion. A receiving transition makes event parameter names available in its guard and effect; sender-receiver data uses `payload`.

On a transition: run the source exit block, the transition effect, then the destination entry block. A final state completes the stateful operation. A received event with no enabled transition in the current state is consumed and ignored. Add an explicit transition if ignoring it is unacceptable.

Use one stateful operation as the receiver for a given member, or one ordinary operation. Do not bind the same member to both: that would create ambiguous delivery.

8. Model realtime constraints

The controller example uses `on every 10 ms`, execution-time bounds and deadlines. A realtime or mixed system requires timing contracts for every used operation and maximum connector latency. Sampled realtime data also needs `maxAge`.

The validator checks positivity, `wcet <= deadline`, and `deadline <= period` for a periodic operation. These are necessary checks, not a scheduling proof. Document the actual RTOS policy, interrupt load, CPU allocation, priorities, clock assumptions, transport timing and measured/analyzed WCET in the implementation project. A production backend must establish them before claiming hard realtime execution.

`serial` is the default unit-local execution policy. `reentrant` requires independent activations and must not mutate shared unit state, including through helpers. An operation waiting for a service response consumes elapsed deadline time even when it consumes no CPU execution time.

9. Separate deployment from architecture

A system can declare execution nodes with kind `mcu`, `process`, or `container`, and `deploy instance.path on nodeName;`. A component deployment applies to descendants unless a more specific descendant deployment overrides it. Each explicit instance path may be deployed only once.

Nodes are allocation targets, not units and not communication shortcuts. Moving a unit to another process cannot make a previously illegal direct data access legal. Network addresses, credentials, task schedules and generated code are outside the 0.1 model.

The mixed gateway shows how one unit explicitly converts sampled values into event notifications. An adapter owns the conversion, timing and failure policy.

10. Organize a larger model

Packages are flat namespaces in 0.1. Write `import Contracts::*;` to use another package's declarations unqualified, or name a declaration `Contracts::Reading`. All relevant files must be loaded together. Two imports exposing the same short name make it ambiguous; use the qualified name.

The editor host should load the whole model workspace. The CLI deliberately loads the files you supply and does not search unrelated directories. Do not put secrets in model payloads, examples or deployment notes.

11. Navigate through views

Surface	Purpose
Browser	Navigate packages → definitions → instances → ports/operations → states
GV - General View	Inspect structure, declarations and their relationships
IV - Interconnect View	Inspect connected ports and component delegation
CV - Case View	Inspect cases and linked requirements
SV - Sequence View	Inspect the partial ordering of interface exchanges in a case
GRV - Grid View	Inspect sortable model, interface, timing and deployment tables

Surface	Purpose
FV - Flow View	Inspect an operation's control flow or owned state transitions

These are specified editor behaviors. The current package supplies the model syntax and language server, not a diagram renderer. View declarations can already be parsed and checked. See graphical notation for the drawing and editing contract.

12. Resolve diagnostics

Diagnostic	Typical fix
`SW030`	Connect immediate child ports; expose/delegate a port across each component boundary
`SW031` / `SW032`	Use the same interface definition and complementary port roles
`SW034`	Connect the port in a closed system, or mark an intentionally unbound port `optional`
`SW050` / `SW051`	Use a local port member and the correct communication verb/direction
`SW052` / `SW055`	Match handler parameters/results and give each incoming member one owner
`SW057`	Make every failure path return or fail
`SW062`	Give competing transitions distinct priorities

A parser error usually means a reserved identifier, missing delimiter, or malformed block. Resolve syntax errors first; semantic checks are skipped on syntactically incomplete documents to avoid misleading secondary errors.

Service fields, repeated consumers and checked traces

Service field models one temperature setting with explicit `get`, `setPoint` and `changed` interface members. The service stores the value privately. Its serial setter publishes through its provided port, which fans out to two required ports on separate panel instances. Each panel owns a distinct displayed value and its own receiving operation.

The getter is reentrant and reads the private-state snapshot taken when it is dispatched. It can safely overlap the serial setter because it does not mutate storage, directly or through a helper. Snapshot implementation is an execution-backend obligation; parsing this example does not simulate it.

A case reply must follow a unique unanswered matching request through its `after` ancestry. Two outstanding requests on the same route/member cannot be disambiguated by drawing one reply lower on the page. Order the pairs or use separate cases. Requests may remain unanswered in a partial trace. A route-valid case still needs an operation-level witness before it counts as verified behavior.

Use the owning package's name as a GV/GRV subject for an inventory. IV takes a component/system; FV takes an operation, including an occurrence path such as `Cabin.settings.store.update`. CV/SV take a case or system. A view cannot use another view as its semantic subject.

If a request returns Void, its required `as` name marks completion but cannot be read as a value. Reserve `failure` for the implicit error-path binding. Write string line breaks with `\n`; raw line breaks, tabs and unsupported escapes inside strings are rejected.

The operational contract defines execution edge cases, and the conformance obligations distinguish validator acceptance from runtime and analysis evidence.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Architecture analysis and SWAML design rationale

This is an original design comparison, not a declaration of compatibility with the referenced standards. Research checked on 2026-10-14. The detailed AUTOSAR comparisons use the accessible R21-11/R19-11 specification material as a stable conceptual baseline; they are not a feature-by-feature review of the latest release.

Comparison

Concern	AUTOSAR Classic	AUTOSAR Adaptive	UML 2.5.1 / SysML 2.0	SWAML decision
Architectural focus	Embedded software components, RTE and BSW	Service-oriented adaptive applications	General software / systems modeling	Software units and components with separate deployment
Contract style	Sender-receiver and client-server interfaces	Methods, events and fields	Interfaces, ports, connectors and behavioral constructs	Four explicit interface kinds; fields expand to operations/events
Behavior ownership	Runnable entities and activation events	Application behavior behind service APIs	UML behaviors/operations; SysML actions, states and usages	Operations own flow/states directly in their unit
Exchange boundary	Ports and runtime integration	Service communication	General-purpose mechanisms can be used in several modeling styles	Cross-unit communication must always use ports/interfaces
Timing	Strong embedded integration context	Distributed execution and communication concerns	Expressible constraints; target analyses vary	Period, WCET, deadline, latency and age; no implied scheduling proof
Deployment	ECU/BSW/RTE integration	Processes/machines and middleware	Deployment/allocation abstractions	Logical nodes and separate target binding
Graphics	Tool-specific architecture views	Tool-specific service views	Broad diagram/view families	Browser + GV/IV/CV/SV/GRV/FV

The table's SWAML column is our design judgment. The following paragraphs identify the source concepts and the choices drawn from them.

AUTOSAR Classic

Classic's application, RTE and basic-software layers separate application software from its integration infrastructure. Its virtual functional bus uses dedicated ports. The software-component template distinguishes sender-receiver data and operation-oriented client-server communication; runnable activation links interface activity to executable behavior. Classic Platform overview, Software Component Template, R21-11, R19-11 runnable/operation specification.

SWAML inference: retain strict port contracts and explicit activation, but make them platform-neutral. A unit is not automatically a runnable, and a component is not automatically an AUTOSAR atomic SWC. A Classic backend could map a suitable unit operation to a runnable, `on every` to timing activation and `on call` to operation activation, subject to target restrictions. WCET and queue semantics must survive the mapping.

Do not infer: SWAML does not supply RTE generation, ECU extract completeness, BSW configuration or ARXML conformance. A valid SWAML model may need additional target constraints before export.

AUTOSAR Adaptive

Adaptive communication management distinguishes service methods, events and fields, with proxy/skeleton roles and subscription behavior. That separation is useful for a contract that contains both RPC and asynchronous notification. Communication Management, R21-11, Communication Management, R19-11.

SWAML inference: a `service` interface combines operation and event members; requires/provides ports capture consumer/provider roles. A field is explicitly represented by getter, setter and changed-event members, with storage private to the provider. Separate `publishSubscribe` contracts also support event-only architecture without forcing an RPC service.

Do not infer: a combined service connector is not automatically SOME/IP, DDS, ara::com, discovery configuration or an Adaptive manifest. Dynamic discovery and subscription lifecycle are deferred rather than approximated by undocumented behavior. Adapter units provide explicit mappings between Classic-style data and service events.

UML

UML supplies rich structural and behavioral modeling: components, interfaces, ports, operations, activities, state machines and interactions. Its breadth permits many modeling styles; a software-architecture discipline must choose which relationships constitute legal runtime exchange. OMG UML 2.5.1.

SWAML inference: preserve familiar components, operation signatures, state/flow and sequence concepts while making the communication boundary a language invariant. Replace free signals and arbitrary cross-object access with interface-member paths. Keep simple value types rather than importing the full UML class/object feature set.

“Unofficial UML 3.0” is a useful aspiration - modern textual software modeling on KerML - but not a version or compliance claim. SWAML is named independently and does not reuse UML certification marks.

SysML 2.0 and KerML 1.0

SysML 2.0 uses KerML as its foundation and provides definition/usage modeling, typed features and behavior constructs. KerML offers general modeling elements suitable for a separate domain language. SysML 2.0, KerML 1.0.

SWAML inference: keep type versus occurrence and a common semantic foundation, but scope the authoring language to software. Operations and states live where they are used, removing a separate performer allocation from the user model. The mapping contract states what kernel normalization must preserve; the reference Langium AST is not advertised as a conforming kernel implementation.

SWAML defines Browser navigation and six view families: General, Interconnect, Case, Sequence, Grid and Flow. Each view projects the same semantic model and retains links to its source declarations and occurrences.

Additional architecture lessons

AADL emphasizes architecture-level analysis of performance-critical properties and real-time embedded systems. SEI introduction to AADL, SEI control-systems modeling guide.

SWAML inference: timing and deployment need first-class declarations, but an operation deadline is not evidence of schedulability. Keep the logical architecture and platform execution assumptions separate. A backend can translate suitable subsets to an analysis model rather than claim the small reference validator performs scheduling analysis.

For service and event-driven software, our design additionally makes queue capacity, overflow, duplicate delivery, partial publication and timeout uncertainty explicit. These are original SWAML semantic choices. An event broker is a modeled intermediary when its behavior matters. Local shared-memory coupling, hidden topic names and transport-specific callbacks cannot erase the interface boundary.

Mapping examples and limits

Source concept	Candidate SWAML representation	Additional evidence needed for an exporter
Classic SW component composition	component definitions and instances	supported target categories and composition rules
Runnable	unit operation	activation, argument, task and scheduling mapping
S/R data element	sampled or queued data member	precise buffering and freshness correspondence
Client/server operation	interface operation + handler	error/result mapping, timeout and invocation policy
Adaptive method	service operation member	target method signature and middleware binding
Adaptive event	service event + publishing/receiving operation	subscription/delivery behavior
Service field	explicit read/write/change members	coherent state and notification protocol
UML activity	operation statement flow	subset restrictions and exception/concurrency mapping
UML state machine	stateful operation	flattening or rejection of unsupported hierarchy/regions
SysML part definition/usage	component/unit definition and instance	software-specific ownership interpretation
Physical/external endpoint	adapter unit with typed ports	how actual I/O enters the implementation

Decisions recorded for 0.1

1. Unit is the smallest interface-protected software boundary. Pure local computation does not require a port.
2. Components compose; units execute. Do not make both own independent execution semantics.
3. Interface signatures are contracts; unit operations provide their implementation.
4. All message/data/event exchange names a local port and interface member.
5. Definitions are reusable, but behavior is owned directly where used. No performers.
6. Publish/subscribe is explicit and supports fan-out without assuming a particular broker.
7. A service can combine requests and events, but implicit fields are avoided.
8. State and flow share FV because both belong to operations.

9. A complete, bounded 0.1 language is preferable to undefined syntax for advanced features. Deferred constructs are listed in the implementation/roadmap documents.
10. Tool claims must distinguish parsing, validation, kernel mapping, execution and graphical editing.

Source handling

Links point to the primary standards/publisher sources. Some AUTOSAR PDF endpoints intermittently failed full downloads during research; their official indexed material and platform pages were available. No standard PDF or proprietary template content is redistributed in this repository. The comparison intentionally focuses on the concepts described above rather than claiming exhaustive standard coverage.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Language tooling and integration

Package contract

The package supplies a reusable language core, command-line tools and a language-server adapter. It is marked `private` to prevent accidental npm publication. Consumers can build it locally and use the documented APIs with an explicitly selected language version.

The source of concrete syntax is swaml.langium. Langium generates AST types, grammar metadata, reflection and TextMate syntax. Generated files and `out/` are ignored; rebuild them after checkout.

The implementation follows Langium's grammar → generated AST → references → validation workflow. Langium grammar reference, reference-resolution guide.

Reproducible commands

SH

```sh
npm ci
npm run build
npm test
npm run validate:examples
npm run check
node out/src/cli/main.js examples/01-hello.swaml
node out/src/server/main.js --stdio
```

Node 22 and 24 are the CI targets; Node 22+ is required. Dependencies are pinned in `package-lock.json`. `npm run build` regenerates the grammar before compiling. Never hand-edit generated files.

The package exports `createSwamlServices` from `out/src/language/swaml-module.js`, and AST declarations from `out/src/language/generated/ast.js`. Example usage after a build:

TS

```ts
import { EmptyFileSystem } from 'langium';
import { parseHelper } from 'langium/test';
import { createSwamlServices } from './out/src/language/swaml-module.js';

const { Swaml } = createSwamlServices(EmptyFileSystem);
const parse = parseHelper(Swaml);
const document = await parse(source, { validation: true });
console.log(document.diagnostics);
```

For multiple files, create all documents first and build them together so that imported declarations are available. CLI arguments are the complete loaded workspace. Definitions in unloaded files cannot resolve.

Services and source ownership

File	Responsibility
swaml-module.ts	Dependency injection and registration
swaml-document-builder.ts	Conservative workspace revalidation for paths and negative lookups
swaml-scope.ts	Local-owner scopes, imported/qualified declarations, ambiguity

File	Responsibility
semantic.ts	Instance/port/member paths, type identities, time conversion
swaml-validator.ts	Model invariants and typed behavior validation
CLI	File loading, diagnostics, exit status
LSP server	Standard language-server transport and startup

Cross-references use Langium references for declarations, functions, local operations, states and nodes. Composite endpoint/value/view paths are strings with explicit SWAML resolution. This keeps encapsulation checks explicit, but **generic rename/find-references/navigation does not cover those strings**. A host must supply path-aware providers before advertising full refactoring.

The current scope provider scans loaded documents for clarity. Every changed/added/deleted document conservatively relinks and revalidates all loaded SWAML models, including string paths and previously absent declarations. This favors correct editor diagnostics over incremental performance. Large-model dependency indexing and cancellation-aware expensive checks remain optimization work. The validator skips semantic checks after parser/lexer errors; built-in diagnostics still report malformed input.

Language-server integration

The server implements the Language Server Protocol over a standard transport. Start it with `node out/src/server/main.js --stdio`; reserve stdout for protocol messages and send diagnostic logging to stderr. Clients supply document URIs, text versions, changes and the complete loaded workspace. They consume diagnostics with source ranges and stable diagnostic codes.

Language hosts must select the supported SWAML edition before loading a document, preserve source ranges, and rebuild affected documents after changes. A formatter or graphical host must apply semantic changes to source and validate the result. File associations, transport startup and presentation are host responsibilities.

Browser hosts

The language core has no Node filesystem dependency. Browser applications can create services with an in-memory filesystem and supply document text directly. The Node CLI and language-server adapter own their filesystem access. A worker-based browser host needs its own transport adapter; test document loading, dependency bundling and import resolution in that environment.

Graphical architecture

Implement a semantic projection layer between Langium and diagrams. It should produce stable node/edge IDs, instance-aware paths, source ranges, declaration references, roles, signatures, diagnostics and presentation kinds. View renderers consume that projection, never parse source text independently.

All edits go through a source-edit service. Layout-only edits update a presentation sidecar. Semantic edits update `.swaml`, then parse/link/validate. Reject silent creation of unmodeled exchanges. Graphical contract.

Kernel backend boundary

Build a separate normalized model from the linked/validated authoring AST, then lower through a versioned SWAML semantic library onto KerML. Keep kernel IDs and authoring IDs traceable. The mapping chapter defines obligations; the JSON inventory is design metadata, not serialized KerML.

Validation and release gates

Before a host consumes a new language-core version, run `npm ci && npm run check`, inspect the spec/grammar changes and update versioned fixtures. A grammar change, even if additive, can reserve an identifier previously used in source. Release notes must explain migration.

Tests cover valid fixtures, illegal interface access, scoping, type errors, QoS, state/timing rules and the LSP transport lifecycle. Broader conformance claims require the additional tests in implementation status.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Implementation status - 0.1.1

The normative draft defines the complete bounded 0.1 language. The repository implements the Langium parser, CLI, language-server foundation and broad static checking. It does not implement a runtime, KerML exporter or graphical editor. See the audit report and conformance obligations.

Implemented and tested

Capability	Evidence
Complete grammar, portable string escapes, generated AST/TextMate syntax	generator, all examples, lexical regressions
Imports, qualified names, ambiguity across declaration kinds and owner-scoped references	multi-document and namespace tests
Interface member/role/signature checks; no cross-unit operation/data bypass	contract and isolation tests
Composition, delegation, required boundary/leaf closure, route uniqueness and cardinality	topology tests
Emitting-operation provenance and unique handler ownership	SW050/SW055 tests
Value typing, records/enums/arrays, expressions, pure functions, Void completion restrictions	values/flow and audit tests
Local calls, acyclic call/containment graphs, transitive reentrant write analysis	call-cycle and helper-effect tests
Explicit QoS, sampled buffering and idempotency declaration checks	QoS tests
Positive safe bounds, WCET/deadline/period consistency and realtime completeness	timing and numeric tests
Stateful signature, transition priorities, graph reachability, finality and unconditional-cycle checks	state tests
Deployment references, case direction/dependencies and unique causal reply correlation	allocation/case regressions
All six view subject categories, package and occurrence contexts	view regressions

Capability	Evidence
Conservative workspace relink/revalidation after change/addition/deletion	editor update tests including newly ambiguous names
LSP initialize/diagnostics/shutdown and CLI exit behavior	integration tests
Production-to-spec/normalized mapping inventory	automatic coverage for all 80 grammar rule/type names

`npm run check` rebuilds, runs tests, validates every example, checks documentation links and executable snippets, and detects missing/duplicate grammar mappings. Mapping coverage is a documentation consistency check, not a KerML validation result.

Limits of static acceptance

The reference core checks the static groups listed in chapter 8; it remains labeled **S, partial V** until an independent conformance suite audits every normative clause and boundary case. It is not a model checker. Acceptance does not establish useful emissions, guard feasibility, case realization, idempotency of implemented effects, termination without runtime failure, schedulability, absence of request cycles, or snapshot support on a particular target.

General arithmetic ranges and computed array bounds are runtime checks. Guarded completion cycles are protected by a specified runtime limit. Activation backlog is bounded by a required execution binding. These are specified execution/analysis obligations, not unimplemented syntax disguised as successful static proof.

String-valued paths now receive conservative dependency invalidation. Path-aware rename, navigation, reference search and persisted editor IDs still need dedicated IDE providers. The all-document rebuild is correct for the loaded workspace but may be expensive on large projects.

Specified, not implemented

Capability	Remaining implementation
KerML K backend	versioned SWAML library, normalized mapper, kernel validation, reverse traces and round-trip tests
Execution E backend	frames, scheduler, state reactions, queues, snapshots, correlation, failures and evidence
Timing/behavioral analysis	target scheduling model, case witnesses, idempotency and safety/liveness evidence
Browser and graphical G workbench	projections, layout sidecars, Properties, validated source edits and undo
Complete IDE refactoring	path-aware rename/find references/navigation and stable identity map
Platform exports	target bindings and serialization such as AUTOSAR ARXML

No K, E or G conformance claim is made. These implementation milestones have complete language-side contracts; they are not necessary to claim that the 0.1 specification draft has defined its syntax and meaning.

Intentionally outside 0.1

Inheritance/redefinition, generics, dynamic object creation, unbounded arrays, reference-valued payloads, hierarchical/orthogonal state regions, fork/join syntax, host-language code blocks, dynamic service discovery, runtime subscribe/unsubscribe, topic wildcards, exactly-once delivery, full requirements algebra and geometry require a versioned language proposal. They cannot be introduced as undocumented backend behavior.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Specification completeness audit - 0.1.1

Date: 2026-10-14. **Scope:** complete the bounded 0.1 language contract and resolve defects found in the initial Langium reference core. Language header remains `swaml "0.1";`.

Findings and resolutions

Issue	Finding	Resolution and evidence
#1	Lexical/value/scope rules were broader in implementation than prose	Portable lexer, cross-kind namespace ambiguity, Void result/failure scope and signed constant index checks; `test/audit.test.ts`
#2	Execution ordering was underspecified and helpers could hide reentrant writes	Operational contract, transitive write analysis, positive read-only and negative helper-chain tests
#3	Reply correlation, reverse publish direction, view subjects and alias closure were incomplete	Unique causal request matching, strict subject categories, required alias checks and duplicate normalized routes; audit tests and service-field example
#4	Composite path and negative lookup diagnostics could become stale across files	Conservative workspace relinking; tests change, restore and delete imported definitions and introduce ambiguous declarations
#5	Mapping inventory omitted syntax families and conformance obligations mixed validity with proof	Normalized model, obligations, all 80 grammar rule/type names mapped and checked

The issues' resolution records link the publication commit and CI evidence. Repository tests supply implementation evidence for static checks; prose-only runtime/kernel behavior is labeled as a contract awaiting a backend.

Completion checklist

Area	Completion boundary
Static architecture	Ownership, definitions/occurrences, ports/interfaces, connectors/delegation, closure and deployment defined
Dynamic architecture	Pure functions, operations, structured flow, local calls, states and activation defined
Communication	Data/RPC/pubsub/service events have provenance, directions, typed payloads, handler ownership, correlation and QoS
Runtime edge cases	Startup, timers, queue overflow, fanout, timeout ties, snapshots, failure and resource binding obligations defined
Scenarios and graphics	Case dependencies/correlation, Browser/GV/IV/CV/SV/GRV/FV subjects and notation/edit rules defined
Kernel foundation	Every syntax family has normalized information and proposed kernel bases/library responsibilities; no false exporter claim

Area	Completion boundary
Developer handoff	Grammar, services, CLI/LSP, examples, user guide, AGENTS and contribution guide agree with the revision
Validation	Reproducible `npm run check`, executable examples, malformed input and cross-file regression coverage

The full gate verifies **87 tests, 8 example models and 80 grammar mapping entries**, plus documentation links and a complete executable guide snippet. CI checks Node 22 and 24. The test suite is a reference-core regression suite; it is not yet an independent certification suite for all V clauses.

Remaining implementation work

KerML serialization/library validation, execution/scheduling, analysis witnesses, graphical editors, persistent IDs, path-aware IDE refactoring and platform exports remain separate implementation milestones. Their absence does not leave the 0.1 syntax undefined, but it limits the tool's conformance claim to S and partial V. See implementation status.

Unsupported language features remain explicit in the specification. This audit does not silently import the rest of UML, SysML, KerML or AUTOSAR. Future features need a versioned proposal and the same syntax, semantics, mapping, evidence and migration checklist.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

ADR 0001 - Ownership and communication boundaries

Status: accepted for SWAML 0.1. Date: 2026-10-14.

The repository owner confirmed that units and components communicate only through interfaces, while local calls and variables can remain internal to a unit. The authoring hierarchy is component → unit → operation, with states directly inside stateful operations. Components may compose other components. Pure functions support local value computation.

An operation is the executable primitive. A contract operation signature and its implementation are separate because the interface must remain reusable across implementations. The handler declaration binds them explicitly. There is no performer or independently allocated action object in the user language.

Events belong to interfaces; publishing statements identify a unit-owned port and member. Each outgoing event/data member has a syntactic emitting operation. Each incoming member has exactly one handler owner. This supplies traceable provenance without pretending to prove runtime reachability.

Consequences: more ports are required when decomposing a component into several units. Shared-memory shortcuts cannot be introduced as “internal implementation” across a modeled unit boundary. Keep tightly coupled helpers inside one unit, or model the shared-state service through a contract.

The view families are General, Interconnect, Case, Sequence, Grid and Flow. Browser provides ownership navigation. Flow View covers operation statement flow and owned-state behavior.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Decision 0002: Complete semantics before claiming complete tooling

Status: accepted for the 0.1.1 draft revision, 2026-10-14.

The specification audit found implementation gaps and ambiguous operational details. The language remains a bounded 0.1 design with direct ownership and interface-only boundaries. This revision keeps the syntax/header and clarifies interpretation while tightening previously incomplete validation.

- A reentrant root and all local helpers use one dispatch-time private-state snapshot and cannot write unit state. Static analysis follows helper calls.
- Initialization installs all handlers/routes before startup dispatch. Lifecycle backlog has an explicit bounded target contract; overflow is observable and stops execution.
- All fanout offers are attempted; a rejected offer cannot undo successful ones. Request timeout wins a response observed at the exact expiry instant.
- State timers start at entry completion and belong to a state-entry generation. Completion is processed before the next external stimulus, with a defined microstep bound.
- Replies infer correlation from one unique unanswered request ancestor. The current grammar has no explicit correlation selector; ambiguous partial traces must be split or ordered rather than guessed.
- Required port status survives every delegation level. Normalized routes reject duplicate leaf pairs.
- Every grammar production has a normative home and a mapping-design entry. These entries do not stand in for an implemented KerML library/exporter.
- Static validity, runtime failures, analysis evidence and IDE work have separate conformance obligations. The tool continues to claim S and partial V honestly.

The stricter lexer rejects raw controls/unknown escapes; Void completions cannot be read; imported names are ambiguous even across target kinds. Models accepted accidentally by 0.1.0 may now receive errors. No valid example from the initial draft requires semantic migration. Keeping the 0.1 header is a draft clarification policy, not permission for silent breaking changes after stabilization.

See issues 1-5 and evidence, the operational contract and normalized model.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

PDF specification and release publication

The complete PDF assembles the normative chapters, graphical notation, complete grammar, semantic inventory, all eight valid models, user guide, architecture comparison, implementation status, decisions and contributor/agent guidance. It has a clickable contents page, section bookmarks, internal and external links, vector figures, selectable numbered code, repeated table headings and embedded original source files.

Version identifiers

The first GitHub release is **0.1.0**, tagged `v0.1.0`. It packages the reviewed **0.1.1 specification/reference-core revision** from the initial draft iterations. The source-language header is **0.1**. The release notes and PDF cover state all three identifiers; the existing draft history is preserved.

The language specification remains a draft. A GitHub release does not add runtime, KerML exporter, graphical editor, AUTOSAR conformance or certification claims.

Rebuild the book

Use Python 3.10+ and DejaVu fonts. On Debian/Ubuntu, install `fonts-dejavu-core`; on other systems, point the builder at a directory containing the DejaVu Sans regular/bold/oblique/bold-oblique and DejaVu Sans Mono TTF files.

SH

```sh
python3 -m venv .venv-pdf
. .venv-pdf/bin/activate
python3 -m pip install -r scripts/pdf-requirements.txt
python3 scripts/build-spec-pdf.py --font-dir /usr/share/fonts/truetype/dejavu
python3 scripts/check-spec-pdf.py
```

The builder reads the repository's source documents directly. Its source list is explicit; unsupported Markdown constructs fail the build instead of disappearing. Mermaid illustrations in the graphical chapter are rendered as corresponding vector figures. The code appendix preserves every grammar line; wrapped continuation rows have no new line number. Exact original bytes remain accessible through embedded attachments.

Outputs are the PDF, a JSON source/hash manifest and a SHA-256 checksum under `output/pdf/`. The checksum identifies the generated artifact; PDF metadata may vary between builds. The manifest allows readers to verify exactly which source content an edition contains.

Quality and release gate

Run `npm run check`, build the PDF, and run the PDF checker. Render all pages with Poppler and inspect them for clipped text, overlapping elements, unreadable tables, broken figures, orphan headings and excessive blank pages. Check dense tables and code at full resolution as well as page contact sheets. Automated extraction cannot replace that visual review.

Commit the reviewed source, builder and final artifacts together. The release must target the reviewed commit, with the PDF, source manifest and checksum attached. Use `--notes-file` for release notes and verify the published tag and assets after upload. Preserve release assets once published; issue a new documented edition for corrections.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Contributing to SWAML

SWAML is developed as a language: specification, grammar, semantics, diagnostics and examples must evolve together. Start with the language specification, ownership decision and implementation status.

Development setup

Use Node.js 22+ and npm 10.2.3+:

```sh
npm ci
npm run check
```

This generates the AST/TextMate files, compiles TypeScript, runs the test suite, validates the examples and checks documentation. Generated files, dependencies and compiled output are not committed. The dependency lockfile is committed.

Propose a language change

Describe the actual modeling problem and a small before/after model. Identify the affected grammar productions, normative clauses, KerML representation, validation rules, view projections and migration behavior. Distinguish a new language feature from a new target-platform binding.

A proposal that adds cross-unit communication must identify the port/interface/member route. A proposal for an event must identify its declaration, emitting operation, receiving owner, delivery policy and failure behavior. A proposal for state or control flow must define ordering, concurrency and termination. Avoid accepting syntax before its semantics exists.

Breaking changes require a new language version and migration notes. Additive keywords can still break existing identifiers; document that impact. Record substantial decisions under `docs/decisions/`.

Implement and test

1. Update the normative prose and grammar together.
2. Update the grammar mapping inventory and classify each obligation as static, runtime, analysis, kernel or graphical. Every rule/type needs a normative home. Regenerate with `npm run langium:generate`; update scoping and validators as needed.
3. Add a valid model and a negative regression that would have caught the original issue. Cross-file changes need imported/qualified and ambiguous-name cases.
4. Update the user guide, affected examples, implementation matrix and changelog.
5. Run `npm run check` and report the result in the pull request.

Tests must verify observable language behavior, not copy the implementation's internal algorithm. Preserve malformed-input recovery: editor diagnostics must not crash the language server. New communication features need tests that attempts to bypass interfaces are rejected.

Pull requests and review

Use a focused branch and describe the user-visible modeling behavior. Include a concise syntax/semantic example, the affected specification clauses, validation evidence and any remaining conformance gap. Do not describe a proposed feature as implemented merely because the grammar accepts it.

Reviewers should check that definitions and instances remain distinct, private data cannot escape through paths, all exchanges remain typed, and diagrams do not invent semantics. Compatibility or migration decisions should be explicit before merging.

Documentation and standards

Link to primary sources for external standards. Write original explanations; do not copy standard PDFs or proprietary specification text into the repository. Preserve the existing MIT license for original code and documentation and retain any applicable third-party notices.

Executable `swaml` code fences must be complete, valid models: the documentation checker parses and validates them. Use inline snippets or `text` fences for deliberate fragments, and put intentionally invalid models under `examples/invalid/` with their expected diagnostic.

Resolve specification issues

Track a concrete contradiction, missing semantic choice or validator defect as a GitHub issue with acceptance criteria. Resolve it with matching normative text, implementation where in scope, examples and regression evidence. Close the issue only when its stated criteria are met; do not close runtime/editor implementation work because a specification paragraph exists. Link the resolution commit or merged pull request and record any remaining conformance limitation.

Report a defect

Include the language/core version, a minimal model, exact CLI command or editor action, observed diagnostics and expected rule. For graphical work include the view ID and semantic subject path. For timing/delivery concerns state the target assumptions; a compiler acceptance result alone is not execution evidence.

Do not include credentials, private service URLs or production payloads in examples. Replace them with minimal synthetic values.

INFORMATIVE / GUIDANCE AND PROJECT STATUS

Instructions for agents working on SWAML

Read first

- `docs/LANGUAGE_SPECIFICATION.md` and the normative chapter affected by the task.
- `docs/IMPLEMENTATION_STATUS.md` to distinguish specified and implemented behavior.
- `docs/spec/06-execution-contract.md`, `07-normalized-model.md` and `08-conformance-obligations.md` for execution, mapping and evidence boundaries.
- `docs/decisions/0001-language-boundaries.md` for the owner's accepted architecture choices.
- `CONTRIBUTING.md` for contribution and validation expectations.

Preserve the language invariants

- Components compose units/components. Units own private data, functions and operations. Operations own statement flow or states directly.
- No performers, separate action allocations, global mutable state or free events.
- Every exchange between distinct units/components uses an owned typed port, interface member and explicit connector/delegation route.
- Local calls/data may stay inside a unit. Never resolve a local operation or function from a different unit as a convenience fallback.
- Outgoing events/data must be tied to emitting operations; incoming members must have one handler owner. Service responses remain correlated through their interface route.
- Keep Browser, GV, IV, CV, SV, GRV and FV. FV covers statement flow and operation-owned states.
- Distinguish definition identity from instance paths. Never make same-type instances share private state.

Source and architecture

The normative concrete grammar is `src/language/swaml.langium`. Prose specifies validity and semantics. Update both when meaning changes. Generated files in `src/language/generated/`, TextMate output and `out/` must not be hand-edited or committed.

Keep the language core free of Node filesystem dependencies; CLI/server adapters own platform I/O. Composite paths currently use explicit semantic resolution and need path-aware IDE services. Do not claim generic Langium rename covers them.

KerML mapping is a proposed contract until a real exporter and kernel validation exist. Do not label Langium AST generation as KerML conformance. Likewise do not call an accepted model schedulable, race-free, executable or graphically editable without the corresponding implementation/evidence.

Required checks

For grammar, semantic or source changes run `npm run check`. Use `npm ci` after dependency changes or on a fresh checkout. Add meaningful positive and negative tests for changed language rules, including malformed input when parser recovery is affected. Report failures honestly and fix regressions within the task scope.

For documentation changes run `node scripts/check-docs.mjs` after a build. Complete `swaml` fences are executable fixtures; deliberate fragments should use inline text or a `text` fence.

Working practices

Prefer small, reviewable changes with stable diagnostic codes. Keep every grammar rule/type covered in `spec/kerml-mapping.json`; the documentation checker enforces coverage. Add a decision record for substantial language changes and update the changelog and implementation matrix. Do not silently widen access scopes to make a fixture pass.

Preserve the existing license, unrelated user edits and repository history. Never add credentials or downloaded proprietary standards. Use primary-source links and original analysis. Follow the user's authorization for GitHub changes; local edits alone do not imply permission to publish a release or alter another repository.

D. External references

Primary-source and project links cited in the included text. The comparisons identify their research baseline; this list does not imply conformance to any external standard.

1. OMG KerML 1.0
https://www.omg.org/spec/KerML/1.0

2. KerML specification
https://www.omg.org/spec/KerML/1.0/PDF

3. Classic Platform overview
https://www.autosar.org/standards/classic-platform/

4. Software Component Template, R21-11
https://www.autosar.org/fileadmin/standards/R21-11/CP/AUTOSAR_TPS_SoftwareComponentTemplate.pdf

5. R19-11 runnable/operation specification
https://www.autosar.org/fileadmin/standards/R19-11/CP/AUTOSAR_TPS_SoftwareComponentTemplate.pdf

6. Communication Management, R21-11
https://www.autosar.org/fileadmin/standards/R21-11/AP/AUTOSAR_SWS_CommunicationManagement.pdf

7. Communication Management, R19-11
https://www.autosar.org/fileadmin/standards/R19-11/AP/AUTOSAR_SWS_CommunicationManagement.pdf

8. OMG UML 2.5.1
https://www.omg.org/spec/UML/2.5.1

9. SysML 2.0
https://www.omg.org/spec/SysML/2.0

10. SEI introduction to AADL
https://www.sei.cmu.edu/library/the-architecture-analysis-design-language-aadl-an-introduction/

11. SEI control-systems modeling guide
https://www.sei.cmu.edu/library/developing-aadl-models-for-control-systems-a-practitioners-guide/

12. Langium grammar reference
https://langium.org/docs/reference/grammar-language/

13. reference-resolution guide
https://langium.org/docs/learn/workflow/resolve_cross_references/

14. #1
https://github.com/voidaliot/swaml/issues/1

15. #2
https://github.com/voidaliot/swaml/issues/2

16. #3
https://github.com/voidaliot/swaml/issues/3

17. #4
https://github.com/voidaliot/swaml/issues/4

18. #5
https://github.com/voidaliot/swaml/issues/5

E. License

MIT License

Copyright (c) 2026 Catalin Ilie

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.